



Projet FPDM Documentation Technique







L'objectif de ce document est de détailler le contenu des codes produits et livrés. Celle-ci est complémentaire au rapport interne présentant en détail le projet et à la documentation utilisateur qui présente comment faire marcher l'interface.

Sommaire

Présentation du projet	3
Contexte	3
Projet	3
Cas d'usage	3
Mise en place rapide du code	4
Module Indexation.....	5
Cas d'usage	6
Choix techniques	6
Environnement de développement	9
Tests des programmes	10
Module Triage patients.....	13
Cas d'usage	14
Choix techniques	15
Environnement de développement	18
Catalogue de fonctions	21
Tests des programmes	30
Module Prédiction durée de séjour.....	35
Cas d'usage	36
Choix techniques	37
Environnement de développement	47
Catalogue de fonctions	50
Tests des programmes	63
IHM et API	71
Cas d'usage	72
Choix techniques	73
Environnement de développement	74
Catalogue de fonctions et classes	76
Commentaires	84

Présentation du projet

Contexte

Ce projet “Faire Parler les Données Médicales” est un projet de 3 semestres réalisés par un groupe de 12 élèves de l’Ecole Centrale de Lille, dans le cadre du travail obligatoire de “projet g1g2”.

Le sujet du projet a été proposé par l’entreprise SIB et le laboratoire CRISAL, qui en sont clients. Le CHU de Lille a également été un client et partenaire du projet. L’encadrement et le suivi ont été effectués par les deux professeurs et chercheurs Madame Ben Othman et Monsieur Hammadi.

Projet

L’entreprise SIB est notamment l’éditeur du logiciel SILLAGE. Présent dans 80% des hôpitaux français, ce logiciel permet aux différents services (urgences, soins, pharmacie, logistique, imagerie) et personnels (soignants, médecins, chirurgiens, pharmaciens, gestionnaires, préparateurs) de transmettre et partager mutuellement les informations nécessaires, de récupérer la liste des médicaments à préparer ou à distribuer à tel étage, d’accéder à l’historique des examens subis par un patient, etc.

Or, la même analyse est faite par les éditeurs et par les utilisateurs de ce logiciel : les informations contenues dans les fiches patient ne sont pas assez exploitées. C’est l’origine de ce projet : il est crucial de faire parler les données médicales, c’est-à-dire d’interpréter et de mettre en évidence certaines données contenues dans les fiches patients. C’est le but de ce projet.

Cet objectif est décliné en trois axes principaux, formant les cas d’usage étudiés.

Cas d’usage

Comme vu ci-dessus, le projet est né du constat que les informations présentes dans les fiches patients n’étaient pas assez exploitées. Ce manque d’exploitation des données est



particulièrement visible et sensible dans trois cas, autour desquels le projet s'est plus particulièrement concentré.

Le premier cas d'usage est l'extraction des données pour aider à l'estimation de l'état de santé ou à la rédaction de comptes-rendus. En effet, les informations connues sur les patients n'ont pas de format universel et peuvent prendre différentes formes (mots-clés, texte, liste, ...) ; trouver les données utiles au corps médical peut rapidement devenir fastidieux et engendrer une perte de temps.

Le deuxième cas d'usage est l'aide à la décision au triage. Le triage est la première étape que subit le patient à son arrivée aux urgences, et a pour but de l'orienter vers le bon service et de lui attribuer un degré de priorité de prise en charge. Cette étape est donc cruciale pour le pronostic vital du patient. Or, il arrive encore régulièrement que des patients soient mal orientés.

Le troisième cas d'usage est une estimation de la durée prévisionnelle de séjour du patient à l'hôpital. Connaître la durée de séjour est un plus à la fois pour les patients et leur entourage pour qu'ils puissent s'adapter, mais aussi et surtout pour le corps médical. En effet, ce dernier peut alors avoir une meilleure anticipation de l'allocation des ressources nécessaires au suivi et au traitement d'un patient.

Mise en place rapide du code

Afin de faire fonctionner le code, un certain nombre de librairies et ressources doivent être préalablement installés. Ces installations ont été réalisées sur le serveur. Le processus de mise en place du code est décrit de manière précise dans la documentation utilisateur.



Module Indexation

Cas d'usage

Notre pôle se concentre sur l'extraction et l'indexation des données médicales afin de faciliter l'estimation de l'état de santé des patients et la rédaction de comptes-rendus. Les informations disponibles sur les patients sont souvent non structurées, prenant diverses formes telles que des mots-clés, du texte libre ou encore des listes. L'absence de format universel complique le travail du corps médical, qui peut rapidement être confronté à une perte de temps significative en cherchant les données nécessaires au diagnostic ou au suivi.

Notre solution vise à structurer ces informations non uniformes, à les indexer de manière efficace et à les résumer pour en extraire les éléments pertinents. Cela permet de réduire la charge cognitive des professionnels de santé, d'améliorer la rapidité d'accès aux données critiques et d'optimiser la prise de décision. En proposant une approche automatisée, notre projet contribue à un gain de temps précieux, tout en améliorant la qualité des soins et la précision des analyses médicales.

Choix techniques

Langage : Python Ce langage a été choisi comme langage de programmation principal, car il était déjà bien maîtrisé par l'ensemble de l'équipe dès le démarrage du projet en Septembre 2023. De plus, Python est largement reconnu et utilisé au sein des communautés scientifique et médicale, ce qui en fait un choix pertinent.

Environnement de collaboration : L'utilisation de Jupyter Notebook a été privilégiée pour le prototypage rapide, les tests sur des LLM et la préparation des datasets d'entraînement. Pour le contrôle des versions et le suivi collaboratif, Git a été adopté. Comme il était de plus important que notre travail et les données soient traitées en local, un serveur mis à disposition par le laboratoire CRISAL (Centre de Recherche en Informatique, Signal et Automatique) a permis de travailler dans un environnement partagé et cohérent pour tous les membres de l'équipe.

Dataset : Le projet s'est appuyé sur le dataset MIMIC-IV (Medical Information Mart for Intensive Care), une base de données américaine librement accessible et conçue pour la recherche en sciences médicales. Maintenu par le MIT, cette base regroupe des données anonymisées de patients admis dans des unités de soins intensifs et couvre une grande variété d'informations médicales.



Ce dataset s'est révélé particulièrement pertinent pour notre pôle, non seulement en raison de sa richesse et de sa diversité, mais aussi parce qu'il contient une section dédiée aux notes rédigées par les médecins, notamment les notes de sortie. Ces dernières, souvent non structurées, constituent un exemple idéal pour notre cas d'usage, qui consiste à extraire et structurer des données utiles à partir de formats hétérogènes. Grâce à ces informations, nous avons pu construire des modèles et des outils adaptés à la problématique de structuration des comptes-rendus médicaux, tout en testant leur applicabilité sur des données réalistes et complexes.

Pour notre projet, nous avons donc exploité la base de données MIMIC-IV et entrepris un travail de sélection des colonnes les plus pertinentes dans les différentes tables disponibles. Cette sélection s'est appuyée sur une étude de l'état de l'art que nous avons réalisée en amont, laquelle nous a permis d'identifier précisément les types d'informations nécessaires à notre cas d'usage, ainsi que sur une volonté de compléter certaines informations manquantes dans la partie note de sortie de la base de données.

Afin de manipuler efficacement ces données, nous avons utilisé la bibliothèque Python *pandas*. Cette bibliothèque a été essentielle pour parcourir et filtrer les multiples tables, extraire les colonnes pertinentes et préparer les données en vue de leur exploitation. Une fois les colonnes sélectionnées, nous avons procédé à des jointures sur les différentes tables de la base afin de consolider les informations en un seul *dataframe*. Ce processus de fusion des données nous a permis de regrouper toutes les informations clés dans un format unique et homogène, facilitant ainsi les étapes ultérieures de modélisation et d'analyse.

Ce travail préparatoire a été crucial pour garantir que les données étaient non seulement pertinentes, mais également correctement organisées, permettant ainsi une exploitation optimale dans le cadre de nos objectifs de recherche et de développement.

Utilisation des modèles de langage (LLM)

Dans notre projet, nous avons fait le choix d'utiliser des modèles de langage (LLM) pour leur capacité exceptionnelle à comprendre et traiter le langage naturel. Ces modèles jouent un rôle central dans l'extraction de mots-clés importants à partir de notes médicales rédigées en texte libre, telles que les observations cliniques ou les notes de sortie. Leur capacité à analyser et structurer des données non formatées est essentielle pour répondre aux besoins spécifiques de notre cas d'usage.

Nous avons opté pour une solution basée sur *llama.cpp* avec le modèle *OpenHermes*, une combinaison qui offre un excellent compromis entre performance, adaptabilité et efficacité.

Pourquoi avoir choisi cette solution ?

1. Performance locale et maîtrise des coûts

L'utilisation de *llama.cpp*, une implémentation légère des modèles de langage, nous permet d'exécuter le modèle directement sur nos machines locales, sans dépendre de solutions cloud coûteuses. Cette approche réduit les coûts tout en garantissant un contrôle total sur le processus d'exécution. Par ailleurs, cela assure une meilleure gestion des données sensibles, un point critique dans le domaine médical.

2. Adaptabilité au domaine médical

OpenHermes est un modèle optimisé pour l'analyse et l'extraction de texte dans des contextes médicaux complexes. Grâce à son réglage spécifique, il est capable d'identifier et de structurer les informations pertinentes dans des documents non formatés comme les comptes-rendus médicaux ou les observations cliniques. Cela en fait un outil particulièrement adapté aux besoins du corps médical, qui gagne ainsi en rapidité et en précision dans l'accès aux données.

3. Légèreté et efficacité

Contrairement à des alternatives comme GPT-4, *llama.cpp* est nettement moins exigeant en termes de ressources matérielles. Cela permet de faire tourner le modèle sur le matériel que nous avons à disposition, tout en respectant les contraintes de temps raisonnables pour le traitement des données. Cette légèreté ne compromet pas pour autant la qualité des résultats obtenus, ce qui fait de cette solution un choix particulièrement pertinent pour des projets nécessitant une efficacité maximale avec des ressources limitées.

4. Temps de traitement optimisé

Étant donné les contraintes matérielles et la nécessité d'une exécution dans des délais acceptables, *llama.cpp* s'est révélé être un choix judicieux. Sa capacité à fonctionner efficacement sur des configurations matérielles standards tout en offrant des performances fiables nous a permis de traiter rapidement les données et d'optimiser les résultats sans ralentir le processus de développement.

Grâce à cette solution, nous avons pu répondre aux exigences techniques et scientifiques de notre projet tout en restant dans des limites réalistes de temps et de ressources.

Environnement de développement

Liste des programmes

- `creation_dossier_medical_non_indexed.ipynb` : créé un dossier médical en txt à partir de la base de données MIMIC4
- `patient_folder.py` : Utilise un LLM pour transformer le fichier médical txt en un fichier indexé en chaîne de caractères avec un format proche du json.
- `patient_folder_multi.py` : Module pour l'extraction de données médicales appliquées à plusieurs rapports.
- `test_reports_data_extraction.py` : Module de test de `patient_folder_multi.py`.

Architecture des dossiers

`creation_dossier_medical_non_indexed.ipynb` utilise la base de données MIMIC4.
`patient_folder.py` utilise l'output de `creation_dossier_medical_non_indexed.ipynb`

Libraries

Les librairies utilisées sont :

Externes :

- `os`
- `sys`
- `pandas`
- `time`

Internes

- `import_mimic_data`
- `llm_communication`

Prérequis

Pour `creation_dossier_medical_non_indexed.ipynb` :

Avoir accès à MIMIC4, renseigner son path dans le module `import_mimic_data`

Pour `patient_folder.py` et `patient_folder_multi` :

Télécharger le modèle de LLM openhermes

Renseigner le path du fichier à traiter ligne 91 et le directory path

Tests des programmes

Protocole

Pour `creation_dossier_medical_non_indexed.ipynb` :

Pour la jointure, vérification qu'il n'y ai pas de pertes ni de duplications de données,

Vérifications du type des données avant et après les transformations effectuées.

Toutes les données non conformes ont été exclues

Vérification manuelle aléatoire des output puis vérification globale de tous les fichiers à l'aide d'un LLM

Pour le test du module :
Test de l'extraction sur deux comptes rendus médicaux extraits de la base de données MIMIC IV.

Prérequis

Pour `creation_dossier_medical_non_indexed.ipynb` :

- La librairie pandas avec ses outils.
- Un LLM fonctionnant en mode “offline” afin de conserver la confidentialité des données.

Pour le test de l'extraction :

- Exécuter le module de test.

Resultats

Nous arrivons à un temps de réponse de 5 minutes et le LLM renvoie dans un format de chaîne de caractères structurée avec les informations principales. Cependant il arrive lors de certaine exécution que le format de renvoi ne soit pas exactement au format JSON rendant impossible la conversion sous forme de dictionnaire de clés. Le modèle ne semble pas avoir halluciné d'information.

Notre groupe s'est néanmoins aussi cantonné au test des prompts à partir de modèles open sources plus conséquents (exemple : Llama 3 70B) en passant par des API (les tests ont donc été réalisés avec des données synthétiques pour des raisons de confidentialité des données). Les résultats produits par le LLM obtenu sont alors au bon format et sont calculés. Ces LLMs pourraient alors être déployés dans une architecture locale plus conséquente pour réaliser cette tâche d'extraction de manière plus performante.

Pour `creation_dossier_medical_non_indexed.ipynb` :

La jointure est correcte sans perte ni ajout de données, les fichiers en output contiennent exactement ce qu'il faut et sont tous uniformes.

Commentaires

Les premiers résultats semblent encourageants mais certaines contraintes liées au LLM viennent limiter l'approche mise en place (notamment dans l'obtention d'un résultat au format JSON).

Il est tout à fait envisageable que l'utilisation d'autres modèles puisse offrir des résultats plus



précis ou mieux adaptés à certaines tâches spécifiques. Toutefois, notre choix a été contraint par des limitations matérielles et par les temps d'exécution élevés observés lors de nos tests. L'exploration de modèles plus performants s'est heurtée à des exigences en ressources que notre environnement ne permettait pas de satisfaire pleinement. Ainsi, notre travail a consisté à trouver un compromis viable entre la qualité des résultats et la faisabilité en termes de calcul, tout en garantissant des temps de réponse acceptables pour une utilisation pratique.

Le test de LLM sur des architectures plus conséquentes montrent la validité de l'approche mise par l'équipe, qui réussit quand même à produire des résultats.



Module Triage patients

Cas d'usage

Le pôle triage patients s'intéresse à l'étape cruciale d'accueil et d'orientation des patients aux urgences au sein de l'hôpital. Tristement célèbre lors du Covid 19, le triage est en effet la première étape que franchissent les patients lors de leur arrivée aux urgences. La personne infirmière d'accueil collecte leurs données vitales, leurs symptômes et motifs de visite, et leur attribue en fonction de ces informations un degré de priorité de prise en charge, ainsi qu'une première orientation éventuelle.

Cette étape d'attribution d'un degré de priorité est cruciale dans le diagnostic vital du patient. Elle influence également la durée de temps que le patient va passer aux urgences (temps d'attente puis temps de guérison), donc est importante pour la gestion des ressources humaines et matérielles pour l'hôpital.

Notre module s'essaie à la détermination d'un degré de priorité à l'aide d'un arbre de décision, ainsi qu'à la prédiction d'une première orientation dans l'hôpital par machine learning.

Choix techniques

LLM

Nous nous appuyons sur le Large Language Model (LLM) Llama pour extraire des données depuis les textes. Nous avons choisi d'utiliser ce modèle car il est accessible gratuitement, disponible en local et connectable à Python. C'est un modèle dit "à poids libre", nous pouvions donc facilement nous en servir pour notre projet d'entraînement de modèle. En outre, ses performances sont bonnes comparées à d'autres modèles testés.

Arbre de décision

Pour déterminer le degré de priorité de prise en charge des patients, nous avons construit un arbre de décision. Nous attribuons 5 degrés de priorité différents, de 1 (réanimation) à 5 (non urgent), en examinant les symptômes et motifs de venue du patient ainsi que son historique. Pour construire les processus de décision, nous nous sommes appuyés sur l'échelle canadienne de triage (décrite à cette adresse : [https://amuq.qc.ca/assets/memoires-et-positions/eTG - L echelle canadienne de triage et de gravite.pdf?phpMyAdmin=i,heTT,BhLKb96mm75DwfLeUjab](https://amuq.qc.ca/assets/memoires-et-positions/eTG_-_L_echelle_canadienne_de_triage_et_de_gravite.pdf?phpMyAdmin=i,heTT,BhLKb96mm75DwfLeUjab)) . Pour extraire les informations nécessaires à partir des documents tests, nous avons fait du prompt engineering à l'aide du LLM llama.

Nous aurions pu également travailler avec une ontologie, mais l'encodage ne nous a pas paru pertinent.

Dataset

Nous avons travaillé avec le dataset MIMIC IV, dont l'accès nous a été donné par le laboratoire Cristal. Cette base de données américaine présente des données recueillies dans des hôpitaux qui permettent de suivre le séjour des patients à l'hôpital. Dans notre cas d'usage, nous nous sommes intéressés aux données des patients arrivant aux urgences pour aider le personnel soignant à réguler la saturation des urgences.

Nous avons également accès à la base de données du CHU de Lille. Nous avons fait le choix de ne pas l'utiliser pour plusieurs raisons. Premièrement, la table est intégralement en français alors que nos mots-clés d'extraction sont tous en anglais. Une traduction de la base de données en anglais ou de nos programmes en français par un LLM était certes

envisageable, mais nous risquions de perdre des informations. Deuxièmement, dans la base de données du CHU, les observations en texte libre au sujet des patients étaient extrêmement confuses, pleines d'abréviations diverses pour les mêmes mots. L'extraction de sens à partir de texte libre est certes le but de notre travail de groupe g1g2, mais nous ne pouvions pas nous appuyer dessus pour entraîner notre modèle.

Au niveau des données exploitées pour la mise en place de la fonctionnalité de prédiction, le choix s'est fait en conservant uniquement les données dont disposent l'hôpital lorsque le patient termine son entretien avec l'infirmière d'accueil. Ces données sont réparties dans la base de données MIMIC dans 6 tables différentes (admissions, patients, services, transferts, triage table, edstays table).

Pour plus de détails sur les informations disponibles sur cette base de données, sont présents [ici](#).

Pour lier les tables entre elles, chaque patient possède un ID et chaque prise en charge possède un ID (que la prise en charge débute aux urgences ou non). Il est donc possible de retracer l'historique d'une prise en charge patient avec toutes les données recueillies au cours de son séjour.

Les types de données utilisées sont surtout des données vitales (saturation en O2, fréquence respiratoire, pouls...) et des informations génériques telles que le sexe, l'âge, etc...

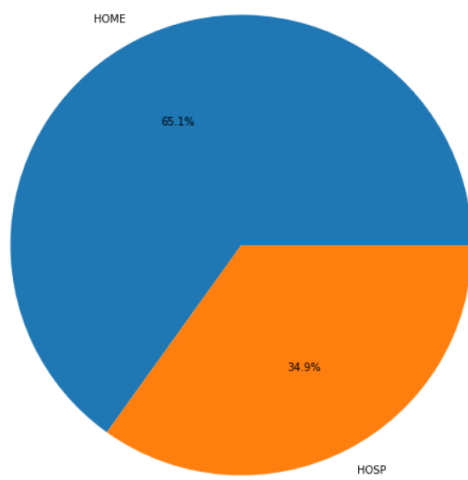
L'information que nous prédisons est la nécessité pour le patient d'être pris en charge au sein de l'hôpital après son séjour aux urgences. Cette information se retrouve dans la table de transferts dans laquelle est renseigné le service dans lequel il est pris en charge après les urgences où "HOME" si il est en mesure de retourner chez lui après son passage aux urgences.

XGBOOST Classifier

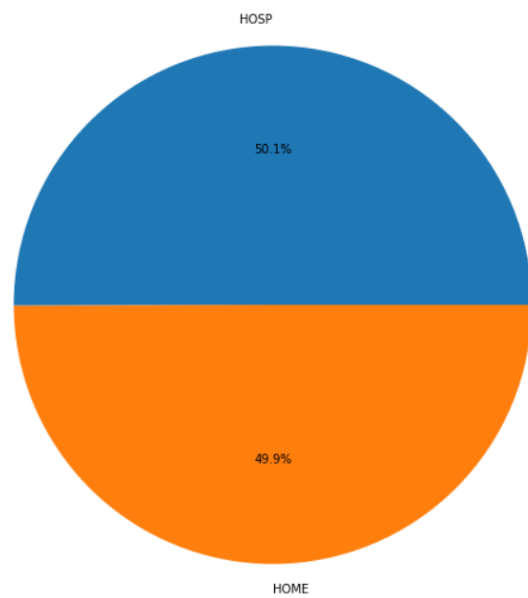
Nous nous sommes servis du modèle de machine learning XG Boost Classifier pour notre deuxième tâche, la prédiction du secteur dans lequel le patient peut être transféré après son passage aux urgences. Nous avons choisi ce modèle en nous appuyant sur des recherches bibliographiques : les articles lus concluaient tous à l'utilisation de XG Boost classifier comme meilleur modèle de machine learning pour cette application. Ce modèle de machine learning est un modèle utilisant le boosting de gradient qui a l'avantage d'être efficace et flexible.

Pour pallier un problème de déséquilibre de notre dataset, effectivement il y a plus de patients capables de retourner chez eux après leur passage aux urgences que de patients ayant la nécessité d'être transféré vers l'hôpital, nous avons eu recours à une solution d'undersampling pour entraîner le modèle. Le principe consiste simplement à retirer aléatoirement des enregistrements de la catégorie dominante pour avoir un nombre d'enregistrement similaire dans chaque catégorie.

Dataset original



Dataset Final



Environnement de développement

Liste des programmes

- Manipulation_table_notbook.ipynb
- Mise en place modèle ML.ipynb
- Main.py
- ChoosePrompt.py
- ExtractionInfosArbreS7.py
- ExtractionInfosArbreS7bis.py
- DecisionTree.py
- test_decision_tree.py

Architecture des dossiers

C:\FPDM_WORKSPACE\MODULETRIAGEPATIENTS

```
| ChoosePrompt.py
| criteria.py
| Decision_tree.py
| test_decision_tree.py
| données pour rapport et audits
| ExtractionInfosArbreS7bis.py
| global_link.py
| prompt_database.csv
| TestsPourRapportEtAudits.py
| traduction_rapports.py
|
|——.ipynb_checkpoints
|   similarites-texte-checkpoint.ipynb
|
|——IA
| | encoding_mapping.pkl
| | entraînement_modele.py
| | main.py
| | manipulation_table_notbook.ipynb
```

- | | Mise_en_place_modèle_random_forrest.ipynb
- | | model_XGBOOST.joblib
- | |
- | |—ipynb_checkpoints
- | | manipulation_table_notbook-checkpoint.ipynb
- | | Mise_en_place_modèle_ML-checkpoint.ipynb
- | | Mise_en_place_modèle_random_forrest-checkpoint.ipynb
- | |
- | |—data
- | | donnees_triage.csv
- | | donnees_triage_unbalanced.csv
- | | donnees_triage_V1.csv
- | | test_donnees_triage_unbalanced.csv
- | |
- | |—test
- | |—2024-12-11_2_categories
- | | | confusion_matrix.png
- | | | notes.txt
- | | |
- | | |—figs
- | | | data_predicted_.png
- | | | data_predicted_HOME.png
- | | | data_predicted_HOSP.png
- | |
- | |—tests_du_module
- | | tests du 7 janvier, avec texte en anglais et réponse du LLM.txt
- | | test_2024_12_06
- | |
- | |—rapports_triage_en_generes
- | | stay_n30795169_repport.txt
- | | stay_n32105905_report.txt
- | |
- | |—rapports_triage_fr_generes
- | | stay_n30795169_repport.txt
- | | stay_n32105905_report.txt
- | |
- | |—tests_extraction_triage
- | | stay_n30795169_repport.json
- | | stay_n30795169_repport08_01_2025.json
- |

```

├──Tests_pole_triage_patients_
|   ├──data_extraction
|   |   ├──test.json
|   |   ├──test_resultat.txt
|   |   └──
|   └──rapports_triage_fr_generes
|       ├──test.txt
|       └──
|   └──[Archives]
|   └──
└──__pycache__

```

Les trois programmes ChoosePrompt, ExtractionInfosArbreS7bis et DecisionTree sont nécessaires à l'arbre de décision.

Le programme ChoosePrompt permet de donner le domaine du texte considéré (y a-t-il évocation de douleur ? le patient relève-t-il du domaine cardiaque, du domaine pulmonaire ? etc.).

Ce premier programme est appelé par ExtractionInfosArbreS7bis. Ce deuxième programme permet d'extraire du texte les informations contenues dedans sous forme d'un dictionnaire, à l'aide de prompts choisis par le premier programme.

Le dictionnaire retourné par le deuxième programme est utilisé par DecisionTree. Ce troisième programme est l'arbre de décision proprement dit et retourne un degré de priorité attribué au patient selon les règles du triage canadien.

Le dossier IA comporte les fichiers relatifs à la prédiction de la nécessité de prise en charge du patient au sein de l'hôpital.

Le premier notebook : manipulation_table_notbook permet de créer la base de donnée sur laquelle le modèle va être entraîné, le notebook Mise_en_place_modèle_random_forrest comporte un test réalisé sur avec un modèle de random forest à la place du modèle XGBOOST. Le fichier encoding_mapping.pkl comporte la table d'encodement des données utilisé pour rendre compatible les données utilisées au fonctionnement du machine learning, model_XGBOOST.joblib comporte le modèle de machine learning utilisé à la suite de son entraînement à l'aide du fichier entraînement_modele.py.

Finalement le programme main.py est celui utilisé dans le livrable, il utilise les fichier du modèle d'IA entraîné et de la table de mapping pour faire les prédictions souhaitées.

Le dossier test comporte les suivis et productions réalisés lors de test pour l'entraînement du modèle de machine learning.

Environnement Git

Pour synchroniser les avancées de ce module avec le reste du projet nous avons mis en place deux branches supplémentaires dans l'environnement github. La première branche (developpement-arbre-de-triage) étant la branche principale du pôle qui était mis à jour régulièrement lors d'avancées sur la fonctionnalité de triage et l'autre (trriage-patients-thomas) était réservée à la fonctionnalité de prédiction qui était majoritairement développée sur le serveur et qui n'a nécessité une branche uniquement à la fin du développement lorsque le modèle était entraîné.

Librairies

Le programme `ExtractionInfosArbreS7bis` importe les librairies python `"sys"`, `"os"`, `"IIm_communication"` et `"json"`.

Pour faire fonctionner les programmes de la partie machine learning, les modules nécessaires sont :

- Pandas
- Joblib
- Pickle
- Os,sys
- Sklearn
- Datetime (pour le suivi des tests)
- Numpy
- Matplotlib
- Seaborn
- xgboost

Prérequis

Le programme `ExtractionInfosArbreS7bis` doit être lancé depuis le répertoire contenant le fichier dont on veut extraire des informations.



Les programmes liés à l'entraînement du machine learning utilisent le fichier `import_mimic_data` pour l'import des données provenant de la base de données MIMIC.

Catalogue de fonctions

Manipulation_table_notbook.ipynb :

`process_select_transfert (group : dataframe) -> DataFrame`

Cette fonction permet de sélectionner le premier transfère après la prise en charge au urgence.

Arguments :

- `group` : C'est un dataframe contenant les informations relative à une même prise en charge

Return :

- La fonction renvoie un dataframe contenant au plus une ligne correspondante au information données plus un colonne indiquant le premier transfert réalisé à la sortie des urgences.

Remarques :

Cette fonction est relativement longue à s'exécuter.

Mise en place modèle ML.ipynb :

`directory_creation(test_name : string) -> None`

Cette fonction met à jour les variables globales du notebook ayant pour objectif de sauvegarder les données du test effectué.

Arguments :

- `Test_name` : Nom du test qui sera aussi le nom du dossier dans lequel les informations sur le test seront stockées. Cette chaîne de caractères ne doit donc pas contenir de caractères spéciaux tels que “,” “'” “\” ...

Remarques :

Les effets de bord de cette fonction sont la création d'un nouveau dossier dans le répertoire "IA/test" et d'un fichier texte à l'intérieur de ce dossier pour y inscrire des informations textuelles tout au long du test.

Main.py :

predictHospitalStay(data : dataframe or dictionary) -> int

Cette fonction utilise le modèle de machine learning XGBOOST du module pour prédire à partir de données patient renseignées par le personnel soignant.

Arguments :

- data : Ensemble des données du patient représenté sous la forme :

```
data_exemple = {  
    'temperature':[45],  
    'heartrate':[11],  
    'resprate':[18.0],  
    'o2sat':[0.2],  
    'sbp':[12],  
    'dbp':[56],  
    'anchor_age':[45065],  
    'gender': ['male'],  
    'arrival_transport': ['ambulance'],  
    'pain': ['high'],  
}
```

Return :

- La fonction renvoie un entier correspondant à la nécessité pour le patient d'être pris en charge ou non (1 nécessaire ; 2 non).

Remarques :

Cette fonction a besoin du modèle de machine learning et du modèle de mapping pour fonctionner.

checkIndex(list_index : pd.Index) -> int

Valider le nom des colonnes de list_index afin de l'utiliser pour prédire le besoin de soins du patient au sein de l'hôpital.

Argumentns :

- - list_index (pd.Index) : ne peut pas être None

Sortie :

- - Validité des données (booléen) : True ==> Les données contiennent les bonnes colonnes ; false ==> Il y a une erreur

ChoosePrompt.py :

Ce programme a pour objectif de comparer un rapport dont on veut extraire les informations avec des rapports de références afin de déterminer quelles sont les informations pertinentes. Pour se faire on s'appuie sur le modèle de vectorisation de texte <https://huggingface.co/sentence-transformers/all-mpnet-base-v2> afin de convertir des textes en vecteurs de VECTOR_SIZE == 768 flottants que l'on compare à l'aide de la similarité cosinus : $sim(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| \times ||\vec{b}||}$ (le résultat obtenu est un flottant entre -1 et 1. Une base de données de rapports de référence (avec des rapports dont on a déterminé les prompts à utiliser) doit être mise en place dans le fichier "prompt_database.csv".

vectorize_text (text : String, model : SentenceTransformer ou None) -> return List[float]

Description de ce que fait la fonction

La fonction permet, à partir d'un texte, de renvoyer un vecteur représentant les mots du texte selon la logique du modèle (par défaut, Sentence Transformer).

Arguments :

- text : texte sous forme de chaîne de caractères que l'on veut vectoriser. Il s'agira du texte en saisie libre dont on veut extraire les informations.
- model (optionnel) : modèle de vectorisation déjà importé (pour éviter de multiples

imports successifs). Par défaut, si l'utilisateur ne spécifie pas de modèle, la fonction importe automatiquement SentenceTransformer.

Return :

- La fonction renvoie le texte vectorisé, c'est-à-dire une liste de flottants.

compare_vectors (vector1 : List[float], vector2 : List[float]) -> return Float

Description de ce que fait la fonction

La fonction permet de comparer deux vecteurs, selon la similarité cosinus.

Arguments :

- vector1 : premier vecteur, représenté par une liste de flottants
- vector2 : second vecteur, également représenté par une liste de flottants, de même longueur que le premier vecteur.

Return :

- La fonction renvoie un flottant compris entre -1 et 1 qui donne la proximité entre les deux vecteurs.

evaluate_similarities (text1 : String, text2 : String, model : SentenceTransformer ou None) -> return Float

Description de ce que fait la fonction

La fonction permet de comparer deux textes, selon la similarité cosinus, grâce aux deux fonctions auxiliaires précédentes. Les textes sont tous les deux vectorisés grâce à la fonction `vectorize_text`, et les deux vecteurs ainsi obtenus sont passés en argument à la fonction `compare_vectors`.

Cette comparaison de deux textes, appliquée au texte en saisie libre dont on veut extraire les données et à des textes de référence, permet de déterminer le domaine pertinent à mobiliser pour cibler l'extraction de données.

Arguments :

- text1 : premier texte, sous forme de chaîne de caractères, par exemple le texte en saisie libre dont on veut à terme extraire des informations.
- text2 : second texte, également sous forme de chaîne de caractères, par exemple un texte de référence contenant tous les mots clés d'un domaine.
- model (optionnel) : modèle de vectorisation déjà importé ou None. Si None, le modèle est importé par la fonction.

Return :

- La fonction renvoie un flottant compris entre -1 et 1 qui donne la proximité entre les deux textes.

vector_str_to_vector (vector_str : String) -> return List[float]

Description de ce que fait la fonction

La fonction permet de convertir la sérialisation d'un vecteur (au format str) en un vecteur (liste de flottants).

Arguments :

- vector_str : représentation sous forme d'une chaîne de caractères d'une liste de flottants.

Return :

- La fonction renvoie une liste de flottants, chaque flottant étant la conversion d'un élément initialement présent dans l'argument de la fonction.

import_reports_database (filename : String, relative_path : String, return_dataframe : Boolean) -> return List

Description de ce que fait la fonction

La fonction permet d'obtenir les rapports médicaux, leur vectorisation et la liste des prompts qui doivent être utilisés pour extraire du texte les informations nécessaires.

Arguments :

- filename : chaîne indiquant le nom du fichier avec les textes de références, par exemple "prompt_database_csv".
- relative_path : chemin relatif du fichier
- return_dataframe (booléen, False par défaut) : booléen qui indique si l'on veut aussi renvoyer un dataframe contenant les informations de la base.

Return :

- Si return_dataframe == False la fonction renvoie une liste de triplets [report, categories, vector] contenant les informations de chaque ligne de la base de donnée :
 - report correspond au rapport de référence (str).
 - categories correspond à une liste de labels correspondant aux catégories de prompts déterminées (clés du dictionnaire prompt_classification_thresholds)
 - vector correspond à la vectorisation de report (liste de VECTOR_SIZE flottants).

vectorize_database_reports (filename : String, relative_path : String) -> None

Description de ce que fait la fonction

La fonction permet de vectoriser les rapports de la base de données non vectorisés et de sauvegarder le résultat dans la base (pour ne pas avoir à refaire le calcul à chaque fois).

Arguments :

- filename : chaîne indiquant le nom du fichier de la base de données, par défaut "prompt_database_csv"
- relative_path : chaîne de caractères donnant le chemin relatif vers le fichier depuis le dossier courant, par défaut la chaîne vide.

Return :

- La fonction ne renvoie rien mais sauvegarde une nouvelle version de la base de données de référence contenant la vectorisation de tous les rapports.

determine_prompts_to_use (report : String) -> return List[String]

Description de ce que fait la fonction

La fonction permet, en utilisant les fonctions auxiliaires ci-dessus, de renvoyer la liste des noms des prompts à utiliser lorsqu'on lui donne en argument un rapport. Pour cela, la fonction commence par appeler le modèle SentenceTransformer puis vectorise le rapport grâce à la fonction `vectorize_text`. Ensuite, la fonction `import_reports_database` permet d'importer les textes de référence auxquels le rapport va être comparé individuellement par la fonction `compare_vectors`. Si le taux de comparaison est supérieur à un score donné dans la liste `prompt_classification_thresholds`, alors on ajoute la catégorie reflétée par le texte de référence concerné à la liste renvoyée en réponse.

Arguments :

- `report` : le rapport à analyser, sous forme de chaîne de caractères

Return :

- La fonction renvoie la liste des noms des prompts utiles à utiliser pour extraire les informations du rapport.

ExtractionInfosArbreS7.py :

Le programme commence par quelques consignes permettant d'importer l'environnement et le LLM, ainsi que décrivant la configuration de travail. Vient ensuite en commentaire le dictionnaire récapitulatif des clés extraites.

On crée ensuite les prompts qui seront passés en consigne au LLM, lui demandant d'extraire les informations du texte libre qui seront utilisées par le programme.

On note la racine du répertoire des documents.

Remarque : ce programme est extrêmement lent, si bien qu'il s'interrompt (1h30 pour le tiers de l'extraction demandée, et le reste ne vient pas même après plusieurs heures d'attente. Les tests avaient été réalisés sur de petits textes de 2 ou 3 lignes).

PromptJson (brut : String, consigne : String) -> return JSON

Description de ce que fait la fonction

La fonction permet d'extraire d'un texte en saisie libre "brut" les informations souhaitées demandées par le prompt "consigne". Ces informations sont renvoyées sous la forme d'un dictionnaire (demandé par le prompt) dans un fichier JSON.

Arguments :

- brut : texte en saisie libre sur lequel le LLM travaillera pour en extraire les informations
- consigne : prompt passé en consigne au LLM ; il demande l'extraction de données dans le texte libre brut, en spécifiant le format du rendu.

Return :

- La fonction renvoie la réponse du LLM dans un fichier JSON

getinformation (reportName : String) -> return JSON

Description de ce que fait la fonction

La fonction permet, à partir du nom d'un fichier, d'en extraire toutes les informations nécessitées par le triage. Ces informations sont renvoyées sous la forme d'un dictionnaire dans un unique fichier json nommé bigdata.

Arguments :

- reportName : nom du fichier texte contenant le texte en saisie libre dont on veut extraire les informations.

Return :

- La fonction renvoie un fichier JSON nommé bigdata contenant les informations demandées par les prompts, sous forme de dictionnaire. Le fichier JSON contient uniquement les informations utiles au triage du patient considéré, les clés pour lesquelles les valeurs sont les valeurs par défaut (pas de problème de santé) sont éliminées.

Remarques :

- La fonction getinformation prend en argument le nom du fichier et va le chercher dans le dossier où étaient stockés nos fichiers d'entraînements. Il faudra modifier le pathDocument dans une version

ExtractionInfosArbreS7bis.py :

Le programme commence par quelques consignes permettant d'importer l'environnement et le LLM et décrivant la configuration de travail. Vient ensuite en commentaire le dictionnaire récapitulatif des clés extraites.

On crée ensuite les prompts qui seront passés en consigne au LLM, lui demandant d'extraire les informations du texte libre qui seront utilisées par le programme. Chaque prompt rassemble les informations d'une catégorie (cardiaque, respiration...). Cela permet au programme de ne donner au LLM que les prompts pertinents, et donc de gagner du temps et de l'espace (voir programme ChoosePrompt.py).

Un dernier prompt vient préciser à quelle catégorie correspond chacun des prompts ci-dessus.

On note la racine du répertoire des documents.

Remarque : ce programme est assez lent (10 minutes environ pour un texte de 3 lignes).

PromptJson (brut : String, consigne : String) -> return JSON

Description de ce que fait la fonction

La fonction permet d'extraire d'un texte en saisie libre "brut" les informations souhaitées demandées par le prompt "consigne". Ces informations sont renvoyées sous la forme d'un dictionnaire (demandé par le prompt) dans un fichier JSON.

Arguments :

- brut : texte en saisie libre sur lequel le LLM travaillera pour en extraire les informations
- consigne : prompt passé en consigne au LLM ; il demande l'extraction de données dans le texte libre brut, en spécifiant le format du rendu.

Return :

- La fonction renvoie la réponse du LLM dans un fichier JSON

get_information_from_str (texte : String, reportName : String, categories_to_prompt : Liste de string) -> return String

Description de ce que fait la fonction

La fonction permet, à partir d'un texte, de son nom et de la liste des prompts utiles, d'extraire du texte toutes les informations nécessitées par le triage (déterminées à partir du programme ChoosePrompt.py). Ces informations sont renvoyées sous la forme d'un dictionnaire python bigdata.

Arguments :

- texte : nom du fichier texte contenant le texte en saisie libre dont on veut extraire les informations.
- reportName : nom du fichier contenant le rapport. Peut aussi ne pas être passé en paramètre (déclaré comme reportName=None lors de la définition de la fonction)
- categories_to_prompt : liste des noms des prompts à passer au LLM.

Return :

- La fonction renvoie un dictionnaire python bigdata contenant les informations demandées par les prompts. Il contient uniquement les informations utiles au triage du patient considéré, les clés pour lesquelles les valeurs sont les valeurs par défaut (pas de problème de santé) sont éliminées.

`get_information (reportName : String, categories_to_prompt : Liste de string) -> return String`

Description de ce que fait la fonction

La fonction permet, à partir du nom d'un texte et de la liste des prompts utiles, d'appeler la fonction ci-dessus (`get_information_from_str`) afin d'extraire du texte toutes les informations nécessitées par le triage (déterminées à partir du programme `ChoosePrompt.py`). Ces informations sont renvoyées sous la forme d'un dictionnaire python bigdata.

Arguments :

- `reportName` : nom du fichier contenant le rapport. Peut aussi ne pas être passé en paramètre (déclaré comme `reportName=None` lors de la définition de la fonction)
- `categories_to_prompt` : liste des noms des prompts à passer au LLM.

Return :

- La fonction renvoie un dictionnaire python bigdata contenant les informations demandées par les prompts. Il contient uniquement les informations utiles au triage du patient considéré, les clés pour lesquelles les valeurs sont les valeurs par défaut (pas de problème de santé) sont éliminées.

DecisionTree.py

Ce programme permet d'attribuer au patient un degré de gravité, à partir des données obtenues par l'infirmière d'accueil (signes vitaux, motif de visite, toute information utile) présentées sous forme de dictionnaire python grâce au programme `ExtractionInfosArbreS7bis`.

Ce programme consiste en une série de fonctions booléennes qui s'intéressent chacune à une ou plusieurs clés du dictionnaire et relèvent sa valeur. Par exemple, `is_cardiac_arrest` renvoie True ou False selon les valeurs des clés `imminent_cardiac_arrest` et `cardiac_arrest`.

Le programme est un arbre de décision parce qu'il définit un arbre à partir d'une classe de nœuds. On crée des nœuds contenant une étiquette et une valeur booléenne à partir des résultats des fonctions ci-dessus. Par exemple, on crée le nœud `cardiac_arrest` est le `Node("cardiac arrest", is_cardiac_arrest)`.

On peut ensuite évaluer la gravité du patient en formant des nœuds pour chaque niveau de gravité. Par exemple, on a le noeud level1 qui porte en étiquette “level 1 : immediate life threatening conditions” et un attribut qui vaut True si l’un des noeuds de la liste [Cardiac_arrest, ..., Respiratory_distress] a la valeur True.

test_patient(patient_data:dict, debug) -> int

Description de ce que fait la fonction

La fonction permet d’effectuer le triage patient à partir de données structurées. Pour se faire, elle fait appel à l’arbre de triage pour évaluer la gravité.

Arguments :

- patient_data (dict) : un dictionnaire contenant les données patients. Les clés doivent être des éléments de la liste présent dans le document “criteria.py”.

Return :

- Un entier entre 1 et 5 évaluant la gravité du cas selon la classification suivante (échelle de triage canadienne)

Niveau de triage	I	II	III	IV	V
Délai	Immédiat	15 min.	30 min.	60 min.	120 min.
Réponse fractile	98%	95%	90%	85%	80%
Taux d'admission attendu	70-90%	40-70%	20-40%	10-20%	0-10%

Tests des programmes

Protocole

Extraction des données (et performances du LLM)

Nous avons sélectionné quelques écrits de la base MIMIC. Nous les avons passés en consigne à notre programme, en y ajoutant une consigne de mesure du temps de réponse.

Arbre de décision

Nous avons vérifié l'exactitude du programme en lui passant en entrée un dictionnaire au format requis.

IA

Nous avons construit un dataset à partir de MIMIC IV en opérant les jointures de tables dont nous avons besoin. Nous avons sélectionné les informations concernant le patient, son historique, et les différents départements de l'hôpital.

La base de données MIMIC IV contient les tables :

- Relatives aux urgences
 - Edstays
 - Triage table
- Relatives aux données générales du patient
 - Admissions
 - Patients
 - Services
 - Transferts

Le fichier d'entraînement du modèle XGBOOST comporte un suivi de test automatisé. Lorsque la première cellule est exécuté, le nom du test est demandé et un dossier comportant les résultats du test est créé dans le dossier : `ModuleTriagePatients/IA/Test/NOM_DU_TEST`. Ce dossier comportera à la fin du test un fichier .txt décrivant l'accuracy, la matrice de confusion et des schémas de répartition des prédictions.

Le fichier main contient 3 fonctions test directement utilisables :

- *testRealData* qui test la fonction *predictHospitalStay* avec un jeu de données issu de MIMIC
- *testRandomData* qui test la fonction *predictHospitalStay* avec un jeu de données exemple fonctionnel
- *testWrongData* qui test la fonction *predictHospitalStay* avec un jeu de données mal ordonnée.

Resultats

Extraction des données (et performances du LLM)

Les tests produisent des résultats comme suit avec le LLM Llama:

```
stay_n30795169_report
Texte en anglais
The patient with ID number 16743399 was admitted on April 2nd, 2021 at 21:02, complaining of fever. They were measuring a
temperature of 103.1°F (39.5°C), a heart rate of 113.0 beats per minute, respiratory rate of 18 breaths per minute and no pain
reported on a scale from 0 to 10.
Réponse du LLM le 7 janvier à 14h30
{ "iss": 21, "glasgow": 8, "severe_headaches": true, "eye_pain": 7, "sudden_thoracic_sharp_pain": false,
"localized_thoracic_pain": true, "associated_symptoms": "dyspnea", "risk_factors": "infarct", "pain_radiation_to": "neck",
"history_of_conditions": "infarct", "abdominal_pain_intensity": 6, "visceral_pain": true, "associated_abdominal_pain_symptoms":
"sudden pain", "pain_level": 8, "colic_pain_intensity": 4, "renal_pain": false, "abdominal_pain": true, "inguinal_pain": true,
"headache_intensity": 9, "headache_onset": "sudden", "associated_headaches_symptoms": "neck_stiffness",
"previous_headache_history": false, "past_pain_experience": "renal colic", "head_pain_level": 8, "pain_level":
}

stay_n32105905_report
Texte en anglais
The patient is male with an ID of 12880928 who was admitted on May 5th, 2021 at 00:49 am. His main complaint is abdominal pain and
he needs to be transferred. He has a normal temperature of 98.3°F, heart rate of 52.0 bpm, respiratory rate of 16 breaths per
minute, and his evaluation of the pain on a scale of 0-10 is 0.
Réponse du LLM le 7 janvier à 14h35
{ "iss": 12880928, "glasgow": 36.8, "severe_headaches": false, "eye_pain": 5, "sudden_thoracic_sharp_pain": true,
"localized_thoracic_pain": true, "associated_symptoms": "dyspnea", "risk_factors": "", "pain_radiation_to": "neck",
"history_of_conditions": "infarct", "abdominal_pain_intensity": 8, "visceral_pain": true, "associated_abdominal_pain_symptoms":
"sudden pain", "pain_level": 7, "colic_pain_intensity": 6, "renal_pain": false, "abdominal_pain": true, "inguinal_pain": true,
"headache_intensity": 4, "headache_onset": "sudden", "associated_headaches_symptoms": "", "previous_headache_history": true,
"past_pain_experience": "fracture", "head_pain_level": 6, "pain_level": 7,
json.)
```

On rencontre alors des problèmes dans le format de sortie du LLM et de certaines valeurs qui sont hallucinées par le LLM. La plus grande limitation provient du temps d'exécution de l'extraction, trop long pour être exploitable dans un contexte réel.

Les mauvaises performances mesurées lors des tests sont vraisemblablement dues à la non sélection des prompts pertinents, à une trop grande longueur de consigne et à des réglages de LLM à améliorer.

Lors de tests avec d'autres LLM (avec des textes écrits à la main et non pas de la base MIMIC pour respecter la confidentialité des données), plus puissants (Llama3.3, DeepSeek R1), les résultats étaient meilleurs : le format json était respecté, les informations étaient cohérentes.

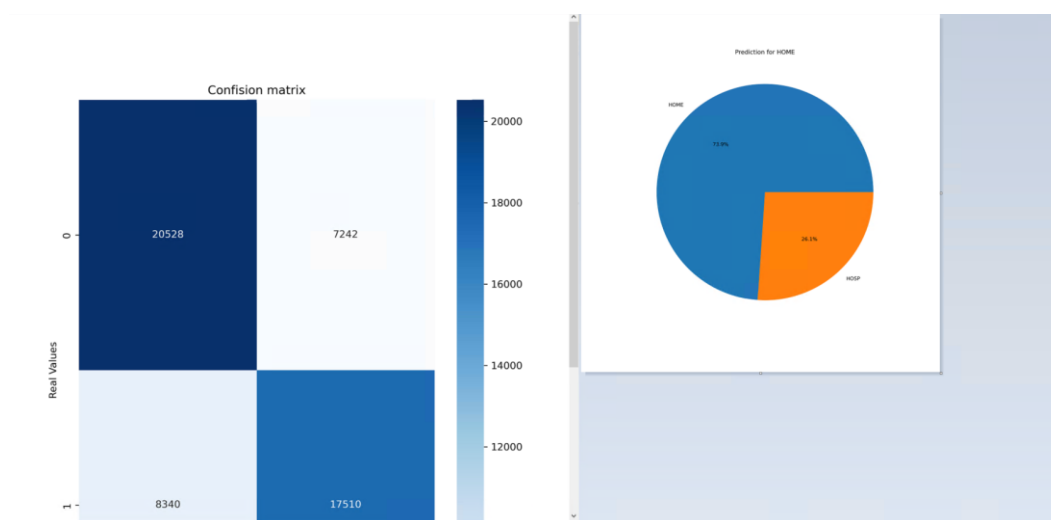
Arbre de décision

Le code est correct.

Les tests réalisés lors de l'entraînement du modèles produisent des résultats comme suit :

figs	16/01/2025 09:27	Dossier d
confusion_matrix	16/01/2025 09:27	Fichier P
confusion_matrix_all_full_dataset	16/01/2025 09:27	Fichier P
encoding_mapping.pkl	16/01/2025 09:26	Fichier P
model_XGBOOST.joblib	16/01/2025 09:27	Fichier J
notes	16/01/2025 09:27	Documer

notes - Bloc-notes
Fichier Edition Format Affichage Aide
Test : 16_01_2025_balanced
Date of the test : 2025-01-16
Accuracy Test : 0.7093994778067885
Accuracy Train: 0.7259133889106474
0 HOME :
HOME : 20528
HOSP : 7242
1 HOSP :
HOME : 8340
HOSP : 17510



Le test *testRealData* :

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModuleTriagePatients\IA> python main.py
0
```

Le test *testRandomData* :

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModuleTriagePatients\IA> python main.py
0
```

Le test *testWrongData* :

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModuleTriagePatients\IA> python main.py
indicearrival_transport incorrect
Traceback (most recent call last):
  File "C:\FPDM_Workspace\ModuleTriagePatients\IA\main.py", line 112, in <module>
    print(testWrongData())
    ~~~~~^
  File "C:\FPDM_Workspace\ModuleTriagePatients\IA\main.py", line 109, in testWrongData
    return predictHospitalStay(data_exemple)
    ~~~~~^
  File "C:\FPDM_Workspace\ModuleTriagePatients\IA\main.py", line 69, in predictHospitalStay
    assert checkIndex(new_data.columns), "not the right lolumns"
    ~~~~~^
AssertionError: not the right lolumns
```




Module Prédiction durée de séjour



Cas d'usage

Le pôle prédiction durée de séjour s'intéresse à la problématique de l'estimation d'une durée de séjour suite à l'admission d'un patient au sein d'un service d'un centre hospitalier.

Cette information pourrait être utile afin d'optimiser la gestion des ressources humaines et matérielles des hôpitaux tout en assurant une prise en charge efficace des patients. En effet, une estimation précise permet d'adapter les plannings des personnels soignants et des lits disponibles. Cela contribue aussi à anticiper les flux d'admissions et de sorties.

Notre module tentera de résoudre cette problématique à l'aide du Machine Learning.

Choix techniques

Langages : Python → Langage de programmation maîtrisé par tout le groupe au début du projet en octobre 2023 et largement utilisé au sein de la communauté scientifique et médicale.

Environnement de collaboration : Jupyter Notebook pour le prototypage rapide du code et la création de dataset d'entraînement, Git pour le contrôle de version, ainsi que le serveur prêté par le laboratoire CRISAL (Centre de Recherche en Informatique, Signal et Automatique) pour partager le même environnement de développement.

Dataset : Pour notre projet, nous avons travaillé avec le dataset MIMIC-IV (Medical Information Mart for Intensive Care), une base de données américaine librement accessible, conçue pour des études en sciences médicales. Cette base est maintenue par le MIT et comprend des données anonymisées provenant de patients hospitalisés dans des unités de soins intensifs. MIMIC-IV couvre un large éventail d'informations :

- Données démographiques des patients (âge, sexe, etc.)
- Données cliniques (résultats biologiques, observations médicales, diagnostics)
- Données administratives (admissions, durées de séjour, motifs d'entrée et de sortie)
- Historique des traitements et interventions.

Ce dataset s'est avéré particulièrement pertinent pour notre projet, car il fournit des indications détaillées sur les séjours hospitaliers, notamment la durée totale de séjour. Nous avons utilisé spécifiquement les données des patients arrivant aux urgences pour construire un modèle d'aide à la régulation des flux d'entrée et de sortie.

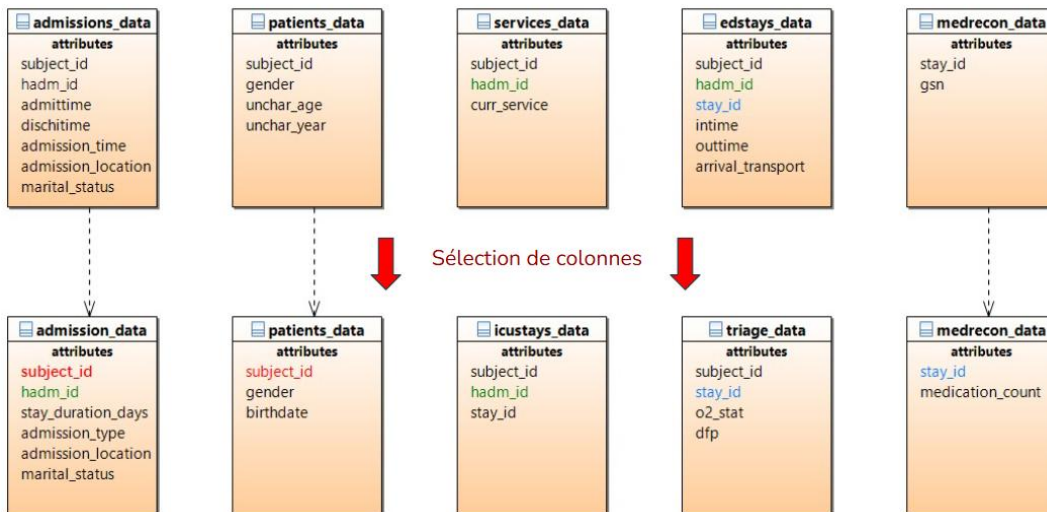
En parallèle, nous avons envisagé d'utiliser la base de données du CHU de Lille pour entraîner un modèle adapté au marché hospitalier français. Travailler avec cette base aurait permis d'élaborer un modèle optimisé pour des hôpitaux en France, grâce à des données contextualisées, notamment :

- Un environnement hospitalier français,
- Des terminologies et des pratiques spécifiques au système de santé français.

Cependant, nous avons décidé de ne pas utiliser cette base car elle ne contenait pas d'informations explicites sur la durée totale des séjours des patients, ce qui était évidemment un critère indispensable pour notre cas d'usage.

Nous avons donc utilisé MIMIC-IV, sélectionné les colonnes pertinentes dans les différentes tables de la base de données en se basant sur l'état de l'art que nous avons réalisé et qui nous a permis d'identifier les informations utiles à la prédiction de la durée de séjour. Et tout cela

grâce à la bibliothèque Panda. Nous avons ensuite réalisé les jointures pour ne créer qu'un seul dataframe sur lequel nous pourrions travailler..



Paramètres retenus :

- Situation de famille (marital_status - string)
- Type d'admission (admission_type - string)
- Lieu de provenance avant admission (admission_location - string)
- Service d'admission (curr_service - string)
- Passage en service d'urgence (icu_visit - boolean)
- Moyen de transport d'arrivée à l'hôpital (arrival_transport - string)
- Pression diastolique (dbp - int)
- Age du patient (age_at_admission - int)
- Nombre de médicaments pris (medications_count - int)
- Temps passé aux urgences avant admission (emergency_department_time_hours - int)

Les trois derniers paramètres ont été créés par nous mêmes à partir des données déjà présentes dans MIMIC-IV, il s'agit de simples calculs d'addition ou de soustraction.

Complétion du dataset

Dans le cadre de l'analyse des données MIMIC-IV, nous avons rencontré des problèmes de données manquantes pour certaines admissions. Ces absences d'informations peuvent être problématiques pour entraîner un modèle de Machine Learning, car de nombreuses techniques, notamment celles basées sur des algorithmes complexes, ne peuvent pas traiter des valeurs manquantes directement. Il était donc crucial de compléter ces données

manquantes de manière appropriée pour garantir la validité et la performance de notre modèle.

1. Identification des Valeurs Manquantes

Au départ, nous avons analysé le dataset pour identifier les variables comportant des valeurs manquantes. Ces variables étaient principalement liées à des informations cliniques ou administratives des patients, comme certains résultats d'examens, des informations sur les traitements ou des caractéristiques démographiques. Après avoir réalisé un premier nettoyage du dataset, il est devenu évident qu'il était nécessaire de compléter ces valeurs pour maintenir l'intégrité de nos analyses et éviter la suppression massive de données, ce qui aurait pu biaiser les résultats.

2. Méthode de Complétion des Valeurs Manquantes : Loi de Probabilité Basée sur la Répartition des Valeurs

Pour traiter ces valeurs manquantes, nous avons opté pour une approche probabiliste basée sur la répartition des valeurs existantes dans le dataset. L'idée était de compléter les valeurs manquantes en suivant une loi de probabilité qui se basait sur la distribution des données déjà présentes pour chaque variable. Cela permettait de conserver la cohérence et la variabilité des données tout en remplissant les informations manquantes de manière plausible.

- Principe : L'approche consiste à modéliser la distribution des données disponibles pour une variable donnée (par exemple, l'âge des patients, leur niveau de créatinine, ou les résultats d'un test biologique). Pour chaque valeur manquante dans une colonne, une valeur est tirée aléatoirement selon cette distribution, en respectant les tendances et les corrélations observées dans le dataset.
 - Par exemple, si une variable suit une distribution normale ou log-normale, la valeur manquante sera remplie en respectant cette même loi de probabilité (avec une moyenne et un écart-type calculés sur les données présentes).
- Pourquoi cette méthode ? :
 - Pragmatisme : Compléter les valeurs manquantes en utilisant des valeurs basées sur des lois de probabilité plutôt que des valeurs arbitraires permet de conserver la structure statistique et les relations entre les différentes variables.
 - Précision : En s'appuyant sur les distributions des données déjà disponibles, on réduit le risque de créer des biais dans les modèles qui pourraient découler d'une complétion aléatoire sans fondement.
 - Cohérence avec les données observées : Cette approche permet de garantir que les nouvelles valeurs respectent les tendances globales des données (par exemple, les relations entre l'âge, le sexe et d'autres variables cliniques).

3. Impact de la Complétion des Valeurs Manquantes sur les Modèles

Une fois ces valeurs manquantes complétées, les données étaient prêtes pour les étapes suivantes du prétraitement et de l'entraînement du modèle. Cette méthode de complétion a

permis de préserver la structure et la variabilité des données, tout en minimisant les biais qui auraient pu survenir si les valeurs manquantes avaient été ignorées ou remplacées par des valeurs fixes (comme la moyenne, la médiane, etc.).

4. Validation et Contrôle

Afin de vérifier que la méthode de complétion des valeurs manquantes était robuste, nous avons procédé à une validation des distributions des variables avant et après complétion. Cette étape a permis de s'assurer que les nouvelles valeurs s'intègrent correctement dans le dataset, sans perturber les relations observées entre les différentes variables.

5. Données dans le dataset

<code>marital_status</code>	<code>string</code>	État civil du patient	'DIVORCED', 'MARRIED', 'SINGLE', 'WIDOWED'
<code>admission_type</code>	<code>string</code>	Type d'admission	'AMBULATORY OBSERVATION', 'DIRECT EMER.', 'DIRECT OBSERVATION', 'ELECTIVE', 'EU OBSERVATION', 'EW EMER.', 'OBSERVATION ADMIT', 'SURGICAL SAME DAY ADMISSION', 'URGENT'
<code>admission_location</code>	<code>string</code>	Localisation avant l'arrivée à l'hôpital	'PHYSICIAN REFERRAL', 'WALK- IN/SELF REFERRAL', 'AMBULATORY SURGERY TRANSFER', 'INFORMATION NOT AVAILABLE', 'CLINIC REFERRAL', 'PROCEDURE SITE', 'PACU', 'TRANSFER FROM HOSPITAL', 'TRANSFER FROM SKILLED NURSING FACILITY',

			'EMERGENCY ROOM', 'INTERNAL TRANSFER TO OR FROM PSYCH'
age_at_admission_filled	integer	Âge du patient lors de l'admission	Un entier, par exemple 34.
curr_service	string	Service actuel du patient	'CMED', 'CSURG', 'DENT', 'ENT', 'EYE', 'GU', 'GYN', 'MED', 'NB', 'NBB', 'NMED', 'NSURG', 'OBS', 'ORTHO', 'OMED', 'PSURG', 'PSYCH', 'SURG', 'TRAUM', 'TSURG', 'VSURG'
icu_visit	integer	Si le patient a visité l'ICU (0 ou 1)	0 (non) ou 1 (oui).
arrival_transport	string	Mode de transport à l'arrivée	'AMBULANCE', 'HELICOPTER', 'OTHER', 'UNKNOWN', 'WALK IN'
medications_count_filled	integer	Nombre de médicaments prescrits	Un entier, par exemple 5.
dbp_filled	float	Tension artérielle diastolique (mmHg)	Un nombre à virgule, par exemple 80.0.
emergency_department_time_hours	float	Temps passé aux urgences (en heures)	Un nombre à virgule, par exemple 2.5.

Post-traitement du dataset

Dans le cadre de notre projet d'estimation de la durée de séjour des patients à l'hôpital, le post-traitement du dataset est essentiel pour préparer les données brutes à l'entraînement du modèle de Machine Learning. Cette étape permet de garantir que les données sont dans un format adéquat pour être traitées efficacement par les algorithmes de prédiction.

1. Standardisation des Données avec StandardScaler

La standardisation est une étape cruciale, en particulier pour les modèles de machine learning sensibles à l'échelle des données, comme les régressions linéaires, les réseaux neuronaux ou encore les SVM (Support Vector Machines). La méthode StandardScaler de la bibliothèque

scikit-learn a été utilisée pour centrer et réduire les variables, ce qui permet de rendre les caractéristiques comparables, notamment lorsque les unités de mesure diffèrent.

- Principe : Le StandardScaler transforme chaque variable en lui attribuant une moyenne de 0 et un écart-type de 1. Cela se fait en soustrayant la moyenne de chaque caractéristique et en divisant par son écart-type..
- Pourquoi l'utiliser :
 - La majorité des modèles de Machine Learning sont sensibles à l'échelle des données.
 - Certaines caractéristiques dans notre dataset, comme les âges ou les résultats biologiques, peuvent avoir des échelles très différentes (par exemple, l'âge peut varier de 0 à 100 ans, tandis que des mesures comme la pression artérielle ou les taux de créatinine peuvent être dans une plage différente).
- Impact sur la performance : La standardisation permet de réduire les biais induits par ces différences d'échelle, rendant les modèles plus robustes et plus performants.

2. Réduction de Dimensionnalité avec PCA (Principal Component Analysis)

Une fois la standardisation effectuée, la réduction de dimensionnalité a été envisagée pour réduire la complexité du modèle et améliorer les performances de prédiction. Le PCA (Principal Component Analysis) est une technique qui permet de projeter les données dans un espace de plus faible dimension tout en conservant autant que possible l'information présente dans les données initiales.

- Principe : Le PCA cherche à identifier des composantes principales, qui sont des directions dans l'espace des données où la variance est la plus grande. L'idée est de réduire le nombre de variables en ne conservant que celles qui expliquent la majorité de la variance. Cela permet de diminuer le nombre de dimensions tout en conservant l'essentiel des informations.
Le PCA fonctionne de la manière suivante :
 - Il centre les données en soustrayant la moyenne.
 - Il calcule les vecteurs propres (directions de la variance maximale) et les valeurs propres (amplitude de la variance).
 - Il projette les données sur les nouvelles dimensions (composantes principales).
- Pourquoi l'utiliser :
 - Le dataset MIMIC-IV contient de nombreuses variables, certaines potentiellement redondantes. En réduisant le nombre de dimensions, le PCA permet de diminuer le risque de sur-apprentissage et de rendre le modèle plus rapide à entraîner.

- Cela permet également de mieux visualiser les données dans un espace réduit de 2 ou 3 dimensions (si nécessaire pour l'analyse).
- Impact sur la performance : Le PCA peut avoir un impact à la fois positif et négatif sur les performances des modèles :
 - Positif : Réduction de la complexité, élimination du bruit et des caractéristiques redondantes.
 - Négatif : Si trop de dimensions sont supprimées, l'information pertinente peut être perdue, réduisant ainsi la capacité du modèle à prédire avec précision.

Dans notre projet, après avoir testé le modèle avec et sans PCA, nous avons constaté que la réduction de dimension permettait de simplifier le modèle et d'éviter l'overfitting tout en maintenant des performances de prédiction relativement similaires.

3. Combinaison de StandardScaler et PCA

Le processus de standardisation est généralement appliqué avant le PCA. Cela permet de garantir que toutes les variables, qu'elles aient une échelle différente, contribuent de manière équivalente à la formation des composantes principales. Sans cette étape, des caractéristiques avec des échelles plus grandes pourraient dominer la construction des composantes principales, et donc fausser le résultat du PCA.

Ainsi, dans notre pipeline de prétraitement, les étapes sont les suivantes :

1. Standardisation des données via StandardScaler.
2. Application de PCA pour réduire la dimensionnalité tout en maximisant l'information conservée.

4. Sauvegarde du Post-traitement avec Joblib

Afin de pouvoir réutiliser le post-traitement sans avoir à le refaire à chaque fois, les objets StandardScaler et PCA ont été sauvegardés à l'aide de la bibliothèque joblib. Cela nous permet de conserver les paramètres appris durant l'entraînement (moyennes, écarts-types pour le StandardScaler, et les composantes principales pour le PCA), afin de les appliquer aux nouvelles données ou lors de futures prédictions.

- Fichiers générés : Deux fichiers joblib ont été créés :
 - composantes principales utilisées pour la réduction de dimensionnalité.scaler.joblib : Ce fichier contient l'objet StandardScaler, qui mémorise les moyennes et écarts-types utilisés pour standardiser les données d'entraînement.
 - pca.joblib : Ce fichier contient l'objet PCA, qui mémorise les

Ces fichiers sont chargés lors du traitement de nouvelles données pour garantir que la transformation est appliquée de manière cohérente avec celle effectuée sur les données d'entraînement.

LLM (Large Language Models) : Ces modèles sont utilisés dans notre projet pour extraire les mots-clés importants des notes médicales rédigées en texte libre. Leur capacité à comprendre le langage naturel est essentielle pour structurer des données non formatées, comme les observations cliniques. Nous avons choisis la solution llama.cpp avec OpenHermes :

- Nous avons utilisé llama.cpp, une implémentation légère de modèles de langage, pour déployer OpenHermes, un modèle optimisé pour les tâches d'extraction et d'analyse de texte médical.
- Pourquoi ce choix ?
 - Performance locale : llama.cpp permet d'exécuter un modèle de langage directement sur des machines locales sans dépendre de solutions cloud coûteuses.
 - Adaptabilité au domaine médical : OpenHermes est finement réglé pour extraire des informations pertinentes dans des environnements médicaux complexes, comme les notes de patients ou les observations cliniques.
 - Légèreté et efficacité : Par rapport à des alternatives comme GPT-4, llama.cpp est moins exigeant en termes de ressources et convient mieux à des projets nécessitant une implémentation économique.

Création des classes de durée de séjour :

Pour améliorer la précision de notre modèle, nous avons choisi de transformer la tâche de prédiction en une classification des patients selon différentes catégories de durée de séjour. Ce choix a permis de simplifier le problème et de rendre les prédictions beaucoup plus fiables.

Les catégories définies étaient les suivantes :

- Catégorie 0 : Entre 0 et 3 jours
- Catégorie 1 : Entre 4 et 9 jours
- Catégorie 2 : Entre 10 et 20 jours
- Catégorie 3 : Entre 21 et 39 jours
- Catégorie 4 : Entre 40 et 69 jours
- Catégorie 5 : Entre 70 et 120 jours
- Catégorie 6 : Plus de 120 jours

Raison du choix des catégories

L'enjeu principal de ce passage aux catégories a été de trouver un compromis entre une précision acceptable et la pertinence des prédictions. En réduisant la granularité de la prédiction en plusieurs catégories, nous avons pu améliorer la précision du modèle tout en conservant suffisamment de détails pour que ces prédictions soient utiles aux professionnels de santé. En effet, pour un hôpital, savoir qu'un patient appartient à une catégorie telle que

"entre 10 et 20 jours" permet d'ajuster les ressources de manière plus efficace que de prédire une durée exacte.

Le processus de mappage des prédictions vers ces catégories a permis d'obtenir des résultats plus précis et plus exploitables dans un cadre opérationnel, en tenant compte des ressources limitées et de la gestion de l'hôpital.

Modèle de prédiction :

Nous avons testé différents modèles comme RandomForestClassifier, XGBClassifier, DecisionTreeClassifier, RNN ou ANN sur le dataset créé précédemment et post-traité.

Test Accuracy: 0.7250463821892393					
Train Accuracy: 0.7766985787969869					
Classification Report (Test Set):					
	precision	recall	f1-score	support	
0	0.78	0.94	0.85	18987	
1	0.43	0.28	0.34	5861	
2	0.31	0.04	0.07	1578	
3	0.59	0.08	0.15	385	
4	0.89	0.28	0.43	113	
5	1.00	0.45	0.62	22	
6	0.67	0.50	0.57	4	
accuracy			0.73	26950	
macro avg	0.67	0.37	0.43	26950	
weighted avg	0.67	0.73	0.68	26950	

XGBC

Test Accuracy: 0.7109833024118738					
Train Accuracy: 0.999944339307581					
Classification Report (Test Set):					
	precision	recall	f1-score	support	
0	0.79	0.89	0.83	18987	
1	0.41	0.33	0.37	5861	
2	0.35	0.15	0.21	1578	
3	0.54	0.21	0.30	385	
4	0.76	0.35	0.48	113	
5	0.91	0.45	0.61	22	
6	0.60	0.75	0.67	4	
accuracy			0.71	26950	
macro avg	0.62	0.45	0.49	26950	
weighted avg	0.68	0.71	0.69	26950	

RandomForestTreeClassifier

Test Accuracy: 0.6558441558441559				
Train Accuracy: 0.9999907232179301				
Classification Report (Test Set):				
	precision	recall	f1-score	support
0	0.80	0.80	0.80	18987
1	0.36	0.35	0.36	5861
2	0.20	0.22	0.21	1578
3	0.23	0.27	0.25	385
4	0.35	0.36	0.36	113
5	0.33	0.45	0.38	22
6	0.22	0.50	0.31	4
accuracy			0.66	26950
macro avg	0.36	0.42	0.38	26950
weighted avg	0.66	0.66	0.66	26950

DecisionTreeClassifier

Train Accuracy: 72.63% Test Accuracy: 72.69%

RNN

Train Accuracy: 72.37% Test Accuracy: 72.37%

ANN

Environnement de développement

Liste des programmes

- **dataset_creation_v2.ipynb** : notebook qui a permis de créer le dataset (dataset_final.csv) à partir de la base de données MIMIC-IV pour entraîner notre modèle
- **set_model_v2.ipynb** : notebook qui a permis de choisir le meilleur modèle de Machine Learning pour notre problématique.
- **extraction_info.py** : script permettant d'extraire les mots importants (informations que le modèle utilise pour faire sa prédiction) dans un compte rendu médical à l'aide d'un LLM.
- **model_training.py** : script permettant d'entraîner le modèle de Machine Learning XGBoostClassifier sur notre dataset et sauvegardent le modèle dans différents fichiers (pca.joblib, scaler.joblib, trained_XGBC_model.joblib) pour être utilisé ultérieurement.

- **prediction_module.py** : script permettant de faire la prédiction de durée de séjour à partir de la chaîne d'informations qu'on lui donne.
- **main.py** : script final communiquant avec l'API et utilisant le fichier `extraction_info.py` et `prediction_module.py`.

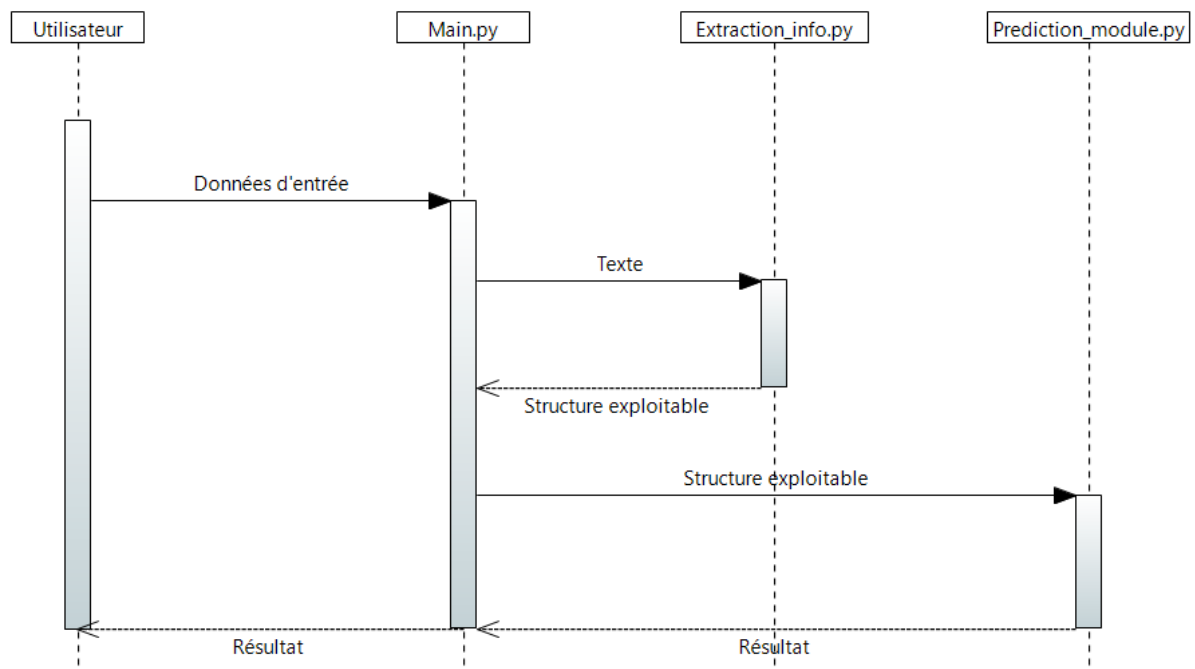
Architecture des dossiers

L'architecture de l'application repose sur une interaction bien structurée entre différents fichiers Python. Le point d'entrée principal est le fichier `main.py`, qui joue le rôle de coordinateur. L'utilisateur commence par fournir une entrée, qui peut être un texte brut ou une liste d'informations. Cette entrée est analysée par `main.py`, qui vérifie si elle est déjà structurée ou nécessite un traitement supplémentaire.

Si l'entrée est un texte, elle est transmise au module `extraction_info.py`. Ce module contient la fonction `extract_info_from_report`, qui extrait les informations pertinentes et les transforme en une structure exploitable (par exemple, une liste ou un dictionnaire). Une fois ce traitement terminé, les données structurées sont renvoyées à `main.py`.

Ensuite, le fichier `main.py` fait appel au module `prediction_module.py`, où se trouvent les principales fonctionnalités liées à la prédiction. La fonction `load_prediction_resources` est utilisée pour charger les ressources nécessaires, telles que le modèle de prédiction, les transformations de données (comme un scaler ou une PCA), et les colonnes de données requises. Ces ressources permettent à la fonction `predict_specific_case` d'utiliser les données structurées pour effectuer une prédiction. Les résultats, comprenant les probabilités, les catégories et la durée prédite, sont ensuite formatés dans un texte lisible par la fonction `display_probabilities_text`.

Finalement, le résultat est affiché à l'utilisateur via le fichier `main.py`. Cette architecture modulaire favorise la clarté et la réutilisabilité, en séparant les responsabilités de traitement de texte, de prédiction et de gestion des interactions utilisateur dans des modules dédiés.



Environnement Git

Plusieurs branches ont été utilisées lors du développement :

- La branche **llamacpp** pour pouvoir utiliser la bibliothèque **llamacpp-python** à la place de **ctransformers** pour interagir avec les LLM.
- La branche **llm_scripts** pour créer des scripts permettant de télécharger des LLM. Cela permet de simplifier le déploiement du projet sur d'autres ordinateurs.

Ces changements ont été intégrés à la branche **main**.

Libraries

- **Panda** : Permet de manipuler des datasets pour notamment faire de la visualisation de données et des opérations basiques sur des bases de données (suppression de colonnes, comptage, etc...).
- **Scikit-learn** : Pour les algorithmes d'apprentissage automatique. Idéal pour des modèles supervisés (régression et classification) avec des workflows bien établis.
- **TensorFlow** : Pour les modèles complexes basés sur des réseaux neuronaux. Adapté pour la création de modèles complexes lorsque les données augmentent en volume et en variété.

- Matplotlib et Seaborn : Pour les visualisations graphiques des résultats. Permet une classification interprétable, essentielle dans le domaine médical pour justifier les décisions.

Ces bibliothèques ont été utilisées car ce sont celles que nous avons pu découvrir lors de nos heures de formations.

Prérequis

1. Environnement Python

Modules requis :

- Pandas
- Matplotlib
- Sys
- import_mimic_data

2. Données et Configuration

Fichiers de données :

- Un fichier JSON de configuration contenant les chemins des bases de données (ex. admissions).
- Fichiers CSV provenant de la base de données MIMIC-IV.

3. Structure du dataset attendu

Les colonnes nécessaires incluent :

- hospital_expire_flag
- admission_location
- age_at_admission
- curr_service
- icu_visit
- ...

Catalogue de fonctions

dataset_creation_v2.ipynb

Fonctions principales dans le notebook

- Chargement des données :
 - Chargement des tables à l'aide de `import_mimic_data.get_table()`.
- Filtrage et nettoyage :
 - Suppression des patients décédés (`hospital_expire_flag == 0`).
 - Restriction aux admissions d'urgence (`admission_location == 'EMERGENCY ROOM'`).
- Création de colonnes calculées :
 - Exemple : Temps passé aux urgences calculé en heures à partir des colonnes `edregtime` et `edouttime`.
- Enregistrement des résultats :
 - Sauvegarde du dataset final dans un fichier CSV (`dataset_final.csv`).

Liste des fonctions créées dans le notebook

assign_age(row, min_age, max_age) -> return int

Assigne un âge aléatoire à une ligne de données en respectant un intervalle donné.

Arguments :

- **row** : Ligne du DataFrame.
- **min_age** : Âge minimum (int).
- **max_age** : Âge maximum (int).

Return

Retourne un entier représentant un âge aléatoire dans l'intervalle spécifié.

assign_random_age(data, column, min_age, max_age) -> DataFrame

Remplit une colonne avec des âges aléatoires dans un intervalle donné.

Arguments :

- **data** : DataFrame cible.
- **column** : Nom de la colonne à remplir (str).
- **min_age** : Âge minimum (int).
- **max_age** : Âge maximum (int).

Return :

Retourne le DataFrame mis à jour.

assign_gender(row, probabilities) -> str

Assigne un genre (homme ou femme) à une ligne en fonction de probabilités définies.

Arguments :

- **row** : Ligne du DataFrame.
- **probabilities** : Dictionnaire contenant les probabilités pour chaque genre (ex {"Homme" : 0.5, "Femme" : 0.5}).

Return :

Retourne une chaîne de caractères représentant le genre assigné.

fill_column_with_empirical_distribution(data, column, distribution) -> DataFrame

Remplit une colonne suivant une distribution empirique donnée.

Arguments :

- **data** : DataFrame cible.
- **column** : Nom de la colonne à remplir (str).
- **column** : Nom de la colonne à remplir (str).
- **distribution** : Dictionnaire représentant la distribution empirique.

Return

:

Retourne le DataFrame mis à jour.

assign_value(row, column, value) -> any

Assigne une valeur fixe à une colonne spécifique d'une ligne donnée.

Arguments :

- **row** : Ligne du DataFrame.
- **column** : Nom de la colonne (str).
- **value** : Valeur à assigner (peut être de n'importe quel type).

Return

:

Retourne la valeur assignée.

fill_column_with_random_values(data, column, values) -> DataFrame

Remplit une colonne avec des valeurs aléatoires choisies parmi une liste donnée.

Arguments :

- **data** : DataFrame cible.
- **column** : Nom de la colonne à remplir (str).
- **values** : Liste des valeurs possibles.

Return :

Retourne le DataFrame mis à jour.

conditional_entropy(x, y) -> float

Calcule l'entropie conditionnelle de x étant donné y.

Arguments :

- **x** : Série ou tableau représentant une variable.
- **y** : Série ou tableau représentant une autre variable.

Return :

Retourne un nombre flottant représentant l'entropie conditionnelle calculée.

theils_u(x, y) -> float

Calcule le coefficient d'incertitude de Theil entre deux variables.

Arguments :

- **x** : Série ou tableau représentant une variable.
- **y** : Série ou tableau représentant une autre variable.

Return :

Retourne un nombre flottant représentant le coefficient de Theil.

entropy(x) -> float

Calcule l'entropie d'une variable.

Arguments :

- **x** : Série ou tableau représentant une variable.

Return :

Retourne un nombre flottant représentant l'entropie calculée.

categoriser_duree_sejour(duree) -> str

Catégorise une durée de séjour en fonction d'intervalles prédéfinis.

Arguments :

- **duree** : Valeur représentant la durée de séjour (int ou float).

Return :

Retourne une chaîne de caractères représentant la catégorie associée à la durée.

set_model_v2.ipynb

Aucune fonction créée ici.

Remarques sur le fichier :

Ce notebook est à considérer comme un environnement sandbox pour comparer les différents modèles sur le dataset et voir quelles opérations de post-traitement du dataset sont pertinentes afin d'avoir à la fin la meilleure précision. Aucun fichier n'est produit ici.

extraction_info.py

isfloat(x) -> bool

Vérifie si une valeur peut être convertie en nombre flottant.

Arguments :

- **x** : Valeur à tester (peut être une chaîne de caractères, un nombre entier, etc.)

Return

:

Retourne True si la valeur peut être convertie en float, False sinon

extract_info_from_report(content) -> list

Extrait des informations structurées à partir d'un rapport médical en texte brut.

Cette fonction analyse un rapport médical en utilisant un modèle de langage (LLM) pour extraire des informations spécifiques comme l'âge, le statut marital, le type d'admission... Elle nettoie et valide les données extraites selon des règles prédéfinies pour chaque champ.

Arguments :

- **content (str)** : Texte du rapport médical à analyser

Return

:

Liste des informations extraites dans l'ordre suivant:

- age_at_admission_filled (int): Âge du patient à l'admission
- dbp_filled (float): Pression artérielle diastolique

- medications_count_filled (int): Nombre de médicaments
- icu_visit (int): Nombre de visites en soins intensifs
- emergency_department_time_hours (float): Temps passé aux urgences en heures
- marital_status (str): Statut marital (DIVORCED/MARRIED/SINGLE/WIDOWED)
- admission_type (str): Type d'admission (ELECTIVE/URGENT/etc.)
- admission_location (str): Lieu d'admission
- arrival_transport (str): Mode de transport à l'arrivée
- curr_service (str): Service médical actuel

Note: Si une valeur ne peut pas être extraite ou validée, "ERROR" est retourné pour ce champ.

Remarques sur le fichier :

Ce script est conçu pour extraire automatiquement des informations médicales structurées à partir de rapports médicaux en saisie libre.

Il utilise un LLM pour extraire 10 informations clés du patient :

- État civil
- Type d'admission
- Lieu d'admission
- Âge
- Service médical
- Passage en soins intensifs
- Mode de transport
- Nombre de médicaments
- Pression artérielle diastolique
- Temps passé aux urgences

Pour chaque champ, le script :

- Envoie une requête spécifique au LLM
- Nettoie et valide la réponse selon des valeurs prédéfinies
- Convertit les données numériques au format approprié

Le script accepte soit un fichier texte (-f) soit du texte direct (-t) en entrée et retourne les informations extraites sous forme de liste formatée.

model_training.py

Aucune fonction créée ici.

Remarques sur le fichier :

Le code reprend les éléments pertinents retenus lors des tests du notebook `set_model_v2.ipynb`. Il commence par charger et prétraiter les données, en supprimant des colonnes inutiles et en encodant des variables catégorielles. Les caractéristiques sont standardisées et réduites en dimension à l'aide de l'analyse en composantes principales (PCA). Un modèle XGBoost est ensuite entraîné sur les données transformées et les objets nécessaires (modèle, scaler, PCA) sont enregistrés pour une utilisation ultérieure. Le modèle est évalué en termes de précision, de rapport de classification et de matrice de confusion, avec des visualisations des probabilités de prédiction et de la matrice de confusion pour analyser ses performances.

prediction_module.py

```
load_prediction_resources() -> return Tuple[Optional[object], Optional[object], Optional[object], Optional[object]]
```

Charge les ressources nécessaires pour faire des prédictions, y compris le modèle entraîné, l'échelle de réduction des données, le transformateur PCA et les noms des colonnes.

Arguments :

- Aucun : La fonction ne prend pas d'arguments directs, elle se base sur les ressources produites par le fichier `model_training.py`.

Return :

Retourne un tuple contenant quatre éléments :

- Le **premier** élément est le modèle entraîné (ou None si le chargement a échoué) qui a été préalablement enregistré dans le fichier `XGBC_trained_model.joblib` lors de l'exécution de `model_training.py`.
- Le **deuxième** élément est le scaler utilisé pour la mise à l'échelle des caractéristiques (ou None si le chargement a échoué) qui a été préalablement enregistré dans le fichier `scaler.joblib` lors de l'exécution de `model_training.py`.
- Le **troisième** élément est l'objet de transformation PCA (ou None si le chargement a échoué) qui a été préalablement enregistré dans le fichier `pca.joblib` lors de l'exécution de `model_training.py`.
- Le **quatrième** élément est une liste des noms des colonnes de caractéristiques (ou None si le chargement a échoué) qui a été préalablement enregistré dans le fichier `X_columns.pkl` lors de l'exécution de `model_training.py`.

Remarques :

1. **Chemin absolu du fichier** : La fonction utilise `os.path.abspath(__file__)` pour obtenir le chemin absolu du script courant. Il est important de s'assurer que le script appelant ce code est correctement situé dans le répertoire attendu, car tout changement de structure de répertoires pourrait entraîner des erreurs dans la récupération des fichiers (modèle, scaler, PCA, etc.). Cela nécessite que l'arborescence du projet reste stable ou qu'un mécanisme de gestion des chemins relatifs soit mis en place si des répertoires externes sont impliqués.
2. **Gestion des exceptions** : La gestion des erreurs utilise un bloc `try-except` pour capturer les erreurs lors du chargement des fichiers. Cependant, cette gestion d'exception est assez générale. Il pourrait être utile d'ajouter des messages d'erreur plus détaillés en fonction de l'exception rencontrée, comme vérifier si les fichiers sont absents, si le format des fichiers est incorrect ou s'il y a un problème de permissions. Cela rendrait le débogage plus facile.
3. **Retour de None en cas d'erreur** : En cas d'échec du chargement de l'un des objets (modèle, scaler, PCA, ou colonnes), la fonction retourne None pour tous les objets. Cela peut être un comportement pratique, mais il peut être plus utile de retourner des informations supplémentaires (par exemple, quel fichier a échoué à se charger) pour mieux comprendre la source du problème.

4. **Dépendance aux fichiers externes** : La fonction dépend fortement de l'existence des fichiers `trained_XGBC_model.joblib`, `scaler.joblib`, `pca.joblib`, et `X_columns.pkl` dans le même répertoire que le script. Si ces fichiers sont déplacés ou renommés, la fonction échouera à charger les ressources nécessaires. Pour améliorer cette situation, il pourrait être judicieux de définir des chemins relatifs ou d'utiliser une configuration externe pour spécifier les emplacements de ces fichiers.
5. **Impact sur d'autres parties du programme** : Toute modification de cette fonction (par exemple, changement du format de stockage des fichiers ou du type d'objet retourné) pourrait affecter directement la partie du programme où ces objets sont utilisés (principalement dans la prédiction des données). Si, par exemple, le format du modèle ou du scaler change, le programme de prédiction devra être mis à jour pour s'assurer qu'il peut toujours interagir correctement avec ces objets.
6. **Portabilité** : Comme la fonction est dépendante de la structure du système de fichiers, elle pourrait ne pas être portable sur différents systèmes d'exploitation ou dans un environnement cloud où les chemins absolus peuvent différer. Il peut être nécessaire d'ajuster la gestion des chemins ou d'utiliser une bibliothèque de gestion des chemins compatible avec plusieurs systèmes d'exploitation, comme `pathlib`.

`predict_specific_case(case: Dict[str, any], model: object, scaler: object, pca: object, X_columns: object) -> return Tuple[Optional[object], Optional[Dict[int, str]], Optional[str]]`

Cette fonction permet de prédire la durée de séjour d'un patient dans un hôpital en fonction de diverses caractéristiques. Elle prend en entrée un dictionnaire contenant les informations spécifiques à un cas, et passe par plusieurs étapes de prétraitement avant de faire une prédiction via un modèle préalablement entraîné. Le processus inclut l'encodage des variables catégorielles, la mise à l'échelle des données, et la réduction de la dimensionnalité à l'aide de PCA. Ensuite, la prédiction est réalisée par le modèle, qui retourne les probabilités associées à chaque classe ainsi que la classe prédite correspondant à une catégorie de durée de séjour.

Arguments :

- **case** : Un dictionnaire contenant les caractéristiques du cas spécifique pour lequel la prédiction est demandée. Les clés doivent correspondre aux colonnes utilisées pendant l'entraînement du modèle.
- **model** : Le modèle pré-entraîné utilisé pour faire des prédictions. Il doit être du même type que celui utilisé lors de l'entraînement et compatible avec les données d'entrée.
- **scaler** : L'objet de mise à l'échelle (scaler) qui a été utilisé pour normaliser les données pendant l'entraînement. Il est appliqué aux nouvelles données avant la prédiction pour

garantir la consistance.

- **pca** : Le transformateur PCA utilisé pour réduire la dimensionnalité des données. Il doit être appliqué aux données après le prétraitement pour les projeter dans un espace de dimensions plus petites.
- **X_columns** : La liste des noms de colonnes utilisées pour entraîner le modèle. Elle garantit que les données d'entrée sont réorganisées dans le même ordre que les colonnes utilisées pendant l'entraînement du modèle.

Return :

Retourne un tuple contenant trois éléments :

- **probabilities** : Les probabilités prédites pour chaque classe de durée de séjour. Celles-ci sont retournées sous forme d'un tableau de probabilités pour chaque classe possible.
- **Categories** : Un dictionnaire qui associe chaque indice de classe à une catégorie de durée de séjour. Par exemple, 0 -> 'entre 0 et 3 jours', 1 -> 'entre 4 et 9 jours', etc.
- **predicted_duration** : La catégorie de durée prédite pour le cas donné. Elle est déterminée à partir de la classe prédite par le modèle et est renvoyée sous forme de chaîne de caractères correspondant à la durée estimée (par exemple, "entre 0 et 3 jours"). Si une erreur survient ou si la classe prédite est inconnue, la valeur "Durée inconnue" est retournée.

Remarques :

1. **Consistance des données d'entrée** : La fonction s'attend à ce que le dictionnaire case contienne les clés exactes correspondant aux colonnes définies dans `specific_case_df`. Toute modification dans la structure des données d'entrée ou l'absence de certaines clés entraînera une erreur ou une mauvaise gestion des données.
2. **Gestion des erreurs** : La gestion des erreurs utilise plusieurs blocs try-except pour capturer les erreurs liées aux étapes de transformation des données (encodage, réorganisation des colonnes, prétraitement, etc.). Il serait utile de préciser dans les messages d'erreur quelle partie spécifique a échoué, par exemple : "Échec de l'encodage des variables catégorielles" ou "Erreur lors de la transformation PCA". La fonction retourne `None` en cas d'erreur, ce qui peut être insuffisant pour un débogage approfondi. Ajouter des logs détaillés ou des messages d'erreur spécifiques serait plus informatif.

3. **Réorganisation des colonnes avec X_columns** : La liste X_columns est utilisée pour réorganiser les colonnes du DataFrame afin de correspondre à l'ordre d'entraînement. Si cette liste n'est pas maintenue à jour (par exemple, si de nouvelles colonnes sont ajoutées ou retirées), cela peut entraîner des erreurs de dimensionnalité et fausser la prédiction. Assurez-vous que X_columns est toujours synchronisé avec les données d'entraînement.
4. **Dépendance aux colonnes catégorielles** : Les colonnes catégorielles (categorical_columns) sont codées en variables binaires à l'aide de pd.get_dummies. Toute modification dans les catégories de ces variables peut nécessiter une mise à jour du code pour s'assurer que les colonnes d'entrée correspondent aux catégories attendues. De plus, l'ordre des colonnes après l'encodage doit être vérifié.
5. **Prétraitement et modèle** : Si des modifications sont apportées au prétraitement des données (scaling, PCA, ou encodage), elles doivent être appliquées de manière cohérente à la fois pour l'entraînement et lors de l'utilisation de cette fonction. Il est essentiel de s'assurer que le scaler et le PCA utilisés ici sont identiques à ceux utilisés lors de l'entraînement du modèle.
6. **Prédiction et gestion des classes** : Le dictionnaire categories mappe les classes prédictives aux intervalles de durée. Si le modèle évolue pour produire de nouvelles classes ou modifier les classes existantes, ce dictionnaire devra être mis à jour pour refléter ces changements.
7. **Impact sur d'autres parties du programme** : Toute modification dans cette fonction (par exemple, ajout ou suppression de colonnes, changement dans la méthode de transformation des données) pourrait impacter d'autres parties du programme qui utilisent les résultats de cette fonction. Les modules ou parties du code qui dépendent de ce format spécifique de données devront être vérifiés et mis à jour en conséquence.
8. **Portabilité** : La fonction utilise pd.get_dummies pour encoder les variables catégorielles, ce qui peut devenir un problème si les catégories dans les nouvelles données sont différentes de celles rencontrées pendant l'entraînement. Cela peut entraîner un désalignement entre les colonnes d'entrée et les colonnes attendues par le modèle, surtout si de nouvelles catégories apparaissent ou si des catégories sont modifiées.

display_probabilities_text(probabilities: Optional[object], categories: Optional[Dict[int, str]], predicted_duration: Optional[str]) -> return str

La fonction prend les probabilités prédites, les catégories associées et la durée de séjour prédite pour générer un texte formaté affichant ces informations de manière lisible. Si l'un des paramètres est manquant (None), un message d'erreur est retourné.

Arguments :

- **probabilities** : Les probabilités prédites pour chaque catégorie (sous forme de tableau ou de liste). Chaque valeur représente la probabilité d'appartenance à une catégorie.
- **categories** : Un dictionnaire qui mappe les indices des catégories aux étiquettes lisibles pour l'humain (par exemple, {0: "entre 0 et 3 jours", 1: "entre 4 et 9 jours"}).
- **predicted_duration** : La catégorie de durée prédite sous forme de chaîne de caractères (par exemple, "entre 0 et 3 jours").

Return :

- **result_text** : Une chaîne de caractères formatée affichant la durée de séjour prédite et les probabilités associées à chaque catégorie. Si un paramètre est None, la fonction retourne un message d'erreur.

Remarques :

1. **Gestion des erreurs** : La fonction vérifie si l'un des paramètres d'entrée est None. Si c'est le cas, un message d'erreur générique est retourné ("Une erreur s'est produite lors du calcul des prédictions"). Cependant, il pourrait être utile de fournir un message d'erreur plus précis indiquant quel paramètre est manquant.
2. **Affichage des probabilités** : Les probabilités sont affichées en pourcentage (avec 2 décimales). Cela peut être utile pour rendre l'information plus lisible, mais cela suppose que les probabilités sont déjà sous forme d'un tableau ou d'une liste à une seule dimension (i.e., chaque ligne dans probabilities[0] correspond à une catégorie). Si la structure des données change, la fonction devra être mise à jour.
3. **Dépendance aux catégories** : La fonction suppose que les indices des catégories dans probabilities correspondent exactement aux indices dans le dictionnaire categories. Si l'ordre des catégories ou la façon dont elles sont indexées change, cela pourrait entraîner un désalignement et une mauvaise interprétation des résultats.
4. **Formatage du texte** : Le texte de sortie est bien formaté, mais cela peut ne pas être suffisant si une sortie plus structurée (par exemple, sous forme de tableau ou de graphique) est requise pour des analyses avancées. Cette fonction est plutôt conçue pour un affichage simple et direct.
5. **Impact sur l'intégration avec d'autres fonctions** : Toute modification dans la structure de probabilities ou des éléments dans categories pourrait casser la fonction. Par exemple, si le format des probabilités ou la manière dont elles sont passées à cette fonction change, cela nécessitera des ajustements dans l'implémentation de cette

fonction.

6. **Portabilité** : La fonction dépend uniquement des entrées sous forme de dictionnaires et de listes. Elle devrait être compatible avec différents systèmes d'exploitation et environnements d'exécution sans nécessiter d'ajustements majeurs.

main.py

```
predict_duration_main(input_data: Union[str, List], display: bool = False) -> return  
Tuple[Union[str, List], List, List]
```

La fonction effectue une prédiction de la durée de séjour pour un cas spécifique en fonction des données d'entrée. Elle prend en charge deux cas distincts pour les données d'entrée :

- **Entrée sous forme de texte** : Si l'entrée est un texte (par exemple, un rapport ou une description brute), la fonction tente d'extraire les informations pertinentes à l'aide de la fonction `extract_info_from_report`. Cela implique une étape de traitement préalable pour transformer le texte en une structure de données exploitable.
- **Entrée sous forme de liste** : Si l'entrée est déjà une liste (par exemple, des données structurées données par l'API), la fonction utilise directement ces données pour effectuer la prédiction sans traitement préalable.

Gestion de l'entrée :

- Si l'`input_data` est un **texte** (chaîne de caractères), la fonction appelle `extract_info_from_report` pour extraire les informations nécessaires. Si l'extraction échoue, un message d'erreur est renvoyé.
- Si l'`input_data` est déjà une **liste**, la fonction l'utilise directement pour la prédiction sans la modifier.

Arguments :

- **input_data** :
 - Type `str` ou `list`. Si `str`, il est interprété comme un fichier ou texte contenant des informations à analyser (extrait avec `extract_info_from_report`). Si `list`, il est directement utilisé pour effectuer la prédiction.
 - Si c'est un texte (`str`), la fonction tente de l'analyser et d'en extraire des informations structurées pour la prédiction.
- **display** : Type `bool`, optionnel, par défaut `False`. Indique si le texte formaté des résultats de prédiction doit être affiché à l'utilisateur.

Return :

- Si une erreur est détectée dans le traitement des données d'entrée (par exemple, un message d'erreur dans `specific_case` comme "ERROR"), la fonction retourne un **tuple contenant `specific_case`, `None`, et `None`**.
- Sinon, elle retourne un tuple contenant :
 - **`specific_case`** : Les données d'entrée traitées et structurées (liste ou texte).
 - **`categories`** : Les catégories des prédictions effectuées.
 - **`probabilities`** : Les probabilités associées à chaque catégorie pour la prédiction.

Remarques :

1. Chargement des ressources :

- a. La fonction appelle `load_prediction_resources` pour charger le modèle, le scaler, le PCA et les colonnes X. Ces ressources sont ensuite utilisées pour effectuer la prédiction via `predict_specific_case`.
- b. Si les ressources de prédiction (modèle, scaler, PCA, colonnes X) sont incorrectes ou absentes, cela pourrait générer une erreur lors de l'exécution de la prédiction.

2. Dépendance aux modules externes :

La fonction dépend de plusieurs autres modules comme `extract_info_from_report`, `load_prediction_resources`, `predict_specific_case`, et `display_probabilities_text`. Si l'un de ces modules ou fonctions est modifié ou supprimé, la fonction `predict_duration_main` risque de ne pas fonctionner correctement.

3. Gestion de l'argument principal :

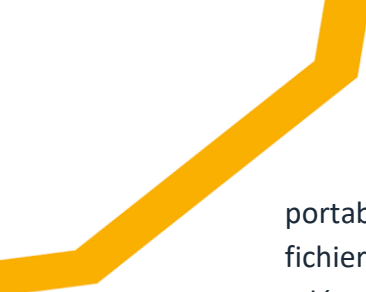
L'utilisation de `argparse` dans le bloc `if __name__ == "__main__":` permet d'exécuter la fonction avec des arguments en ligne de commande. L'argument `input` est traité comme une chaîne de caractères et est converti en une liste si nécessaire.

4. Impact sur l'intégration avec d'autres fonctions :

- a. Si le format des données d'entrée change (par exemple, si `input_data` devient un autre type d'objet), cela pourrait nécessiter des ajustements dans la fonction, en particulier dans la logique de traitement des erreurs et la conversion de `input_data`.
- b. Les fonctions comme `extract_info_from_report`, `predict_specific_case`, et `display_probabilities_text` doivent être compatibles avec les types de données renvoyés par cette fonction.

5. Portabilité :

Le script, lorsqu'il est exécuté en ligne de commande avec `argparse`, est



portable tant que les bibliothèques nécessaires sont installées et que les chemins de fichiers sont correctement configurés. Les dépendances externes doivent être gérées adéquatement.

Tests des programmes

Protocole

Programme dataset_creation_v2.ipynb

Execution du notebook

- Charger le fichier de configuration JSON.
- Exécuter le notebook dans un environnement Jupyter Notebook pour générer dataset_final.csv.

Programme extraction_info.py

Ouvrir la console > Pointer la console vers le dossier qui contient le fichier extraction_info.py
> Exécuter ensuite la commande "python extraction_info.py -f "dossier_patient.txt" ". avec dossier_patient.txt un compte rendu médical de votre choix.

Programme prediction_module.py

Ouvrir la console > Pointer la console vers le dossier qui contient le fichier test_prediction_module.py > Exécuter ensuite la commande "python test_prediction_module.py". Cela permet de vérifier que le module de prédiction fonctionne bien sur des cas spécifiques, vous pouvez modifier le cas du patient en changeant les valeurs du dictionnaire patient_data dans le script en respectant bien les valeurs possibles (vu au début de cette documentation).

Programme main.py

Ouvrir la console > Pointer la console vers le dossier qui contient le fichier test_prediction_module.py > Exécuter ensuite la commande "python main.py" suivi de la chaîne.

Prérequis

Description of how to reproduce the tests.

Programme dataset_creation_v2.ipynb

1. Données d'entrée

Les fichiers CSV contenant les données brutes du dataset MIMIC-IV.

2. Environnement d'exécution

- Jupyter Notebook.

- Modules requis correctement installés.

Programme extraction_info.py

1. Bibliothèques et modules requis

- **os** (standard Python) : Utilisé pour gérer les opérations liées au système d'exploitation sur les fichiers.
- **sys** (standard Python) : Utilisé pour gérer les chemins d'accès.
- **argparse** (standard Python) : Utilisé pour gérer les arguments en ligne de commande.
- **llm_communication** : Utilisé pour communiquer avec le LLM.

Vous devez vous assurer que ces fonctions sont correctement définies et importées.

2. Données d'entrée

- Fichier texte via **-f**
- Texte direct via **-t**

3. Environnement d'exécution

- **Python et bibliothèques nécessaires** : Vous devez disposer d'une version récente de Python (par exemple Python 3.7 ou plus) et des bibliothèques nécessaires à l'exécution du module **llm_communication**, tel que **ctransformers** (pour utiliser le LLM).
- **Configuration de llm_communication** : Assurez-vous que votre fichier **llm_parameters.json** se situe bien à la racine du projet, qu'il soit configuré correctement et que le fichier des poids du LLM choisi soit au bon endroit.

Programme prediction_module.py

1. Bibliothèques et modules requis

- **prediction_module** : Il est impératif que le module **prediction_module** existe et contienne les fonctions suivantes :
 - **load_prediction_resources()**
 - **predict_specific_case(patient_data, model, scaler, pca, X_columns)** :
 - **display_probabilities_text(probabilities, categories, predicted_duration)**

Vous devez vous assurer que ces fonctions sont correctement définies et importées.

2. Fichiers de ressources nécessaires

- **Modèle, scaler et PCA** : Le modèle, le scaler et le PCA doivent être chargés depuis des fichiers externes. Ces fichiers doivent être présents sur le système et être compatibles avec la fonction **load_prediction_resources()**. Par exemple, les fichiers **trained_model.joblib**, **scaler.joblib**, et **pca.joblib** pourraient être nécessaires.
- **Fichier contenant les colonnes (X_columns)** : Ce fichier est nécessaire pour s'assurer que les données patient sont correctement formatées et traitées avant la prédiction.

3. Données d'entrée

- **Format des données patient** : Le dictionnaire `patient_data` doit respecter le format attendu par la fonction `predict_specific_case()`. Assurez-vous que chaque clé du dictionnaire correspond à un champ requis par la fonction de prédiction et que les types de données sont corrects.
 - Par exemple, les valeurs des clés comme `'marital_status'`, `'admission_type'`, et `'curr_service'` doivent être des chaînes de caractères, tandis que des valeurs comme `'age_at_admission_filled'`, `'dbp_filled'`, et `'emergency_department_time_hours'` doivent être des nombres.

4. Environnement d'exécution

- **Python et bibliothèques nécessaires** : Vous devez disposer d'une version récente de Python (par exemple Python 3.7 ou plus) et des bibliothèques nécessaires à l'exécution des fonctions, telles que `joblib` (pour charger les fichiers de modèle et scaler), et toute autre bibliothèque de machine learning (comme `sklearn`) qui pourrait être utilisée par le modèle.
- **Accès aux fichiers externes** : Si le modèle, le scaler et le PCA sont stockés localement, assurez-vous que les fichiers sont accessibles depuis le répertoire où le script est exécuté, ou modifiez les chemins d'accès pour les adapter à votre système de fichiers.

Programme `main.py`

1. Bibliothèques et modules requis

- **argparse** (standard Python) : Utilisé pour gérer les arguments en ligne de commande.
- **typing** (standard Python) : Nécessaire pour les annotations de type (e.g., `Union`, `List`, `Optional`).
- **Modules spécifiques au projet** :
 - `extraction_info` : Doit contenir la fonction `extract_info_from_report`, qui extrait les informations nécessaires d'un texte d'entrée.
 - `prediction_module` : Doit inclure les fonctions suivantes :
 - `load_prediction_resources()`
 - `predict_specific_case(specific_case, model, scaler, pca, X_columns)`
 - `display_probabilities_text(probabilities, categories, predicted_duration)`

2. Ressources externes nécessaires

- **Modèle et ressources de prédiction** :
 - Le modèle, le scaler, le PCA, et les colonnes nécessaires doivent être disponibles dans des fichiers externes et accessibles via la fonction `load_prediction_resources`. Par exemple :
 - Fichier du modèle (e.g., `model.joblib`).

- Fichier du scaler (e.g., scaler.joblib).
- Fichier du PCA (e.g., pca.joblib).
- Fichier décrivant les colonnes attendues (X_columns).

3. Environnement Python

- Version Python : **Python 3.7 ou plus récent** est recommandé pour assurer la compatibilité avec les bibliothèques utilisées (e.g., typing, argparse).
- Bibliothèques supplémentaires (installables via pip si nécessaire) :
 - joblib : Nécessaire si les fichiers du modèle, scaler, ou PCA sont stockés au format Joblib.
 - sklearn (scikit-learn) : Peut être nécessaire si le modèle ou le prétraitement a été construit avec cette bibliothèque.
- Si vous utilisez un environnement virtuel (recommandé), assurez-vous que toutes les dépendances sont correctement installées dans cet environnement.

Resultats

Programme dataset_creation_v2.ipynb

Le fichier dataset_final.csv contient uniquement les colonnes filtrées et les données nettoyées.

Programme extraction_info.py

Avec le fichier en entrée :

“The patient is a single male who was admitted to the emergency room via ambulance on July 9th at 4:30 PM. He was 33 years old and had a body mass index of 26.5. The patient's primary diagnosis was appendix appendicitis, with no secondary diagnoses. On July 10th at 10 AM, he underwent a laparoscopic appendectomy which was completed successfully by provider ID 98765 at facility ID 456. The procedure cost \$5643.0. The patient stayed in the hospital for two and a half days before being discharged home on July 11th at 2 PM. His length of stay in the emergency department was 15.45 hours, and he took four medications during his stay.”

On obtient le résultat ci-dessous :

```
(FPDM_python_env) PS C:\FPDM Workspace\ModulePrediction> python .\extraction_info.py -f .\rapports_dds_en_generees\admission_n21511832_report.txt
Extracted profile: ['SINGLE', 'DIRECT EMER.', 'ERROR', 33, 'ENT', 0, 'AMBULANCE', 4, 'ERROR', 15.45]
```

Programme prediction_module.py

Pour le dictionnaire suivant :

```
# Step 2: Define patient data
patient_data = {
    'marital_status': 'WIDOWED',          # Marital status
    'admission_type': 'DIRECT EMER.',      # Inlet type
    'admission_location': 'AMBULATORY SURGERY TRANSFER', # Place of origin before admission
    'age_at_admission_filled': 63,         # Patient's age
    'curr_service': 'CSURG',               # Current service
    'icu_visit': 1,                        # Visit to the intensive care unit
    'arrival_transport': 'HELICOPTER',      # Arrival mode
    'medications_count_filled': 5,         # Number of medications currently taken by patient
    'dbp_filled': 79,                      # Diastolic blood pressure
    'emergency_department_time_hours': 0.1 # Time spent in emergency
}
```

⇒ On obtient :

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModulePrediction> python test_prediction_module.py
Durée de séjour prévue : entre 4 et 9 jours

Probabilités par catégorie :
- entre 0 et 3 jours: 20.37%
- entre 4 et 9 jours: 69.57%
- entre 10 et 20 jours: 6.71%
- entre 21 et 39 jours: 3.29%
- entre 40 et 69 jours: 0.05%
- entre 70 et 120 jours: 0.01%
- Plus de 120 jours: 0.00%
```

Le patient a de fortes chances (70%) de rester hospitalisé entre 4 et 9 jours.

Programme main.py

- Texte n°1 :

"The patient is a married male who was admitted to the hospital through the emergency room on January 1st, 2017 at 6 PM. He was 60 years old at the time of admission and was seen by the medical service. The patient had no visits to the ICU during his stay. He arrived at the hospital via walking in and was taking 27 medications at the time of admission. His diastolic blood pressure was 80.0, indicating that he spent a significant amount of time in the hospital. The patient had an emergency department time of 9.45 hours before being admitted to the medical service. He stayed in the hospital for a total of 13.6 days before being discharged on January 14th, 2017 at 10:30 AM. His discharge disposition was home, meaning he was well enough to be released without needing further care or observation. The patient's admission source was a primary care provider. During his stay, the patient had 15 diagnoses and 6 procedures performed."

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModulePrediction> python main.py "The patient is a single male who was admitted to the emergency room via ambulance on July 9th at 4:30 PM. He was 33 years old and had a body mass index of 26.5. The patient's primary diagnosis was appendix appendicitis, with no secondary diagnoses. On July 10th at 10 AM, he underwent a laparoscopic appendectomy which was completed successfully by provider ID 98765 at facility ID 456. His distolic blood pressure bdp was 85.0. The patient stayed in the emergency department during 15.45 hours, and he took four medications during his stay."
['SINGLE', 'ELECTIVE', 'AMBULATORY SURGERY TRANSFER', 33, 'ENT', 0, 'AMBULANCE', 4, 85.0, 'ERROR']
```

⇒ Présence de 'ERROR', c'est au pôle API de jouer pour compléter la section manquante et nous renvoyer une liste complète !

- Texte n°2 :

"The patient is a married male who was admitted to the emergency room via ambulance on July 9th at 4:30 PM. He was 33 years old and had a body mass index of 26.5. The patient's primary diagnosis was appendix appendicitis, with no secondary diagnoses. His distolic blood pressure bdp was 85.0. The patient stayed in the emergency department during 15.45 hours, and he took four medications during his stay."

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModulePrediction> python main.py "The patient is a married male who was admitted to the emergency room via ambulance on July 9th at 4:30 PM. He was 33 years old and had a body mass index of 26.5. The patient's primary diagnosis was appendix appendicitis, with no secondary diagnoses. On July 10th at 10 AM, he underwent a laparoscopic appendectomy which was completed successfully by provider ID 98765 at facility ID 456. His distolic blood pressure bdp was 85.0. The patient stayed in the emergency department during 15.45 hours, and he took four medications during his stay."
Durée de séjour prévue : entre 0 et 3 jours

Probabilités par catégorie :
- entre 0 et 3 jours: 93.78%
- entre 4 et 9 jours: 5.59%
- entre 10 et 20 jours: 0.54%
- entre 21 et 39 jours: 0.07%
- entre 40 et 69 jours: 0.03%
- entre 70 et 120 jours: 0.00%
- Plus de 120 jours: 0.00%
```

⇒ Durée de séjour prévue : entre 0 et 3 jours

Probabilités par catégorie :

- entre 0 et 3 jours: 96.39%
- entre 4 et 9 jours: 3.02%
- entre 10 et 20 jours: 0.47%
- entre 21 et 39 jours: 0.09%
- entre 40 et 69 jours: 0.02%
- entre 70 et 120 jours: 0.00%
- Plus de 120 jours: 0.00%

- Et si on donne une liste avec toutes les informations :

"['MARRIED', 'ELECTIVE', 'WALK-IN/SELF REFERRAL', 60, 'MED', 0, 'UNKNOWN', 27, 80.0, 9.45]"

```
(FPDM_python_env) PS C:\FPDM_Workspace\ModulePrediction> python main.py "['MARRIED', 'ELECTIVE', 'WALK-IN/SELF REFERRAL', 60, 'MED', 0, 'UNKNOWN', 27, 80.0, 9.45]"
Durée de séjour prévue : entre 0 et 3 jours

Probabilités par catégorie :
- entre 0 et 3 jours: 74.17%
- entre 4 et 9 jours: 21.60%
- entre 10 et 20 jours: 3.85%
- entre 21 et 39 jours: 0.35%
- entre 40 et 69 jours: 0.03%
- entre 70 et 120 jours: 0.00%
- Plus de 120 jours: 0.00%
```

⇒ Durée de séjour prévue : entre 0 et 3 jours

Probabilités par catégorie :

- entre 0 et 3 jours: 74.17%
- entre 4 et 9 jours: 21.60%
- entre 10 et 20 jours: 3.85%
- entre 21 et 39 jours: 0.35%
- entre 40 et 69 jours: 0.03%
- entre 70 et 120 jours: 0.00%
- Plus de 120 jours: 0.00%

A l'aide des probabilités associées à chaque classe de durée de séjour, le personnel soignant peut avoir une idée de la fiabilité de la réponse donnée par le modèle.



IHM et API



Cas d'usage

L'objectif de l'API est de fournir une interface de programmation permettant l'utilisation des différents modules à partir de paramètres fournis par l'utilisateur. L'API instaure alors un système de requêtes permettant d'interroger les différents modules avec différents types de données. Afin de gérer les accès à l'API, celle-ci inclut aussi un système de permissions incluant des clés d'API ayant certaines permissions.

Le projet inclut aussi une IHM qui se présente sous la forme d'un gestionnaire de dossiers patients informatisés connecté à une petite base de données médicale. L'objectif de cette IHM est de fournir une interface permettant une utilisation plus conviviale de l'API et démontrant comment les modules du projet FPDM peuvent être intégrés dans un projet plus complexe.

Choix techniques

Le code de l'interface et de l'IHM ont été réalisés en utilisant la librairie python **Django** (<https://www.djangoproject.com/>), permettant le développement d'applications Web détaillées, ainsi que les langages de programmation web **HTML**, **CSS** et **Javascript** pour la création de l'interface. L'ensemble est alors exécutable en faisant tourner un serveur local, géré par Django et permettant l'hébergement de l'interface et de l'API.

Le stockage des clés d'API et des permissions est réalisé dans une base de données **SQL Lite**.

Une petite base de données médicale a été mise en place au travers de fichiers **.csv** et d'un programme permettant d'interagir avec la base. L'objectif de cette base est d'être liée à l'interface pour montrer ce qui pourrait être réalisable dans une architecture plus importante. Le code pourrait être modifié pour connecter l'interface à une architecture médicale plus importante.

L'équipe a fait le choix de gérer l'interface et l'API au travers de Django car cette solution était à la fois adaptée à la mise en place de l'IHM et de l'API. L'équipe avait aussi envisagé les solutions technologiques suivante qui ont été abandonnées par la suite :

- **Flask** : librairie Python permettant la création de petites applications Web, plus rapide que Django mais plus limité dans la création d'applications plus limitées.
- **FastAPI** : librairie Python optimisée pour la création d'API mais ne permettant pas la gestion de l'interface.

Environnement de développement

Le code se présente sous la forme d'un projet Django **fpdm_interface_api** incluant deux applications **api** et **user_interface**. L'architecture du code reprend alors l'architecture générée automatiquement par Django. L'ensemble des codes sont disposés dans le dossier **api_interface**.

Architecture des dossiers

(la présente architecture ne fait pas figurer certains codes générés automatiquement par Django mais qui n'ont pas été modifiés par l'équipe.

api_interface

|__ fpdm_interface_api : projet Django

|__ settings.py : fichier de paramétrage du projet Django

|__ urls.py : liste des urls de l'interface et de l'API

|__ api : ensemble du code lié à l'API (application Django)

|__ urls.py : liste des urls de l'API

|__ models.py : description de l'architecture de la base de données de gestion
des clés d'API

|__ forms.py : description des contenu des formulaires d'utilisation de l'API (pour
la validation des données)

|__ views.py : fonctions de gestion des requêtes de l'API

|__ user_interface : ensemble du code lié à l'interface (application Django)

|__ urls.py : liste des urls de l'API

|__ forms.py : description des contenu des formulaires de l'UI

|__ views.py : fonctions de gestion des requêtes de l'interface

|__ static : codes des pages statiques de l'interface

|__ **medical_database_example** : *BD médicale de test et programme d'interaction avec la base*

|__ **patient_information.csv** : *base de données patients*

|__ **reports_list.csv** : *base de données documents*

|__ **interact_with_database.py** : *programme pour interagir avec la BD médicale.*

|__ **test_interact_with_database.py** : *fichier de test des fonctions du module interact_with_database*

|__ **db.sqlite3** : *base de données de gestion des clés d'API*

|__ **generate_first_key.py** : *programme pour la génération d'une première clé d'API*

Libraries

Les librairies suivantes ont été mises en place pour faire fonctionner les programmes :

- **django** (5.1.3) : gestion de l'interface et de l'API
- **requests** (2.32.3) : interrogation de l'API
- **secrets** : génération de clés d'API aléatoires (aléatoire adapté à la cryptographie)
- **bcrypt** (4.2.0) : hachage des clés d'API
- **pandas** (2.2.3) : gestion données
- **csv** : lecture des fichiers csv de la base de test
- **datetime, time** : gestion des dates
- **os, sys**

Librairies déjà installées lors d'une installation standard de python.

Prérequis et utilisation

L'équipe vous renvoie à la documentation utilisateur qui décrit comment mettre en place et exploiter l'interface utilisateur et l'API.



Catalogue de fonctions et classes

Codes “urls.py”

Les codes de la forme **urls.py** indiquent quelles sont les vues (fonction de gestion des requêtes) à appeler en fonction des urls saisies par l'utilisateur.

```
urlpatterns = [
    path("", views.homepage_login),
    path("logout", views.logout),
    path("extract_data", views.extract_data_ui),
    path("extract_data/extract/", views.extract_request),
    path("predict_duration", views.predict_stay_duration_ui),
    path("predict_duration/predict/", views.predict_stay_duration_request),
    path("patient_triage", views.patient_triage_ui),
    path("patient_triage/triage/", views.patient_triage_request),
    path("patients", views.patients),
    path("patients/", views.patients),
    path("documents", views.view_document),
    path("documents/add_commentary/", views.add_commentary),
    path("documents/import_commentary/", views.import_commentary),
    path("documents/delete_document/", views.delete_document),
]
```

- Le code **urls.py** du dossier **fpdm_interface_api** indique quels fichiers **urls.py** sont à utiliser en fonction de l'url entrée par l'utilisateur. En particulier :
 - L'ensemble des urls de la forme “api/...” est géré par le dossier **urls.py** du dossier **api**.
 - L'ensemble des autres requêtes est géré par le code **urls.py** du dossier **user_interface**.
- Le code **urls.py** des dossiers **api** et **user_interface** indiquent quelles vues utiliser en fonction de la requête demandée.

Remarque : dans ces fichiers on indique une url “relative” à laquelle vient s'ajouter la forme des urls indiquées dans le dossier **fpdm_interface_api**. En particulier il faut ajouter **api/** à l'ensemble des urls du code **api/urls.py**.

Ces codes peuvent être modifiés pour changer le routing d'urls. Cependant la modification des urls doit être reportée dans l'ensemble des vues (codes **views.py**) ainsi que dans l'ensemble des pages HTML dans le dossier **static**.

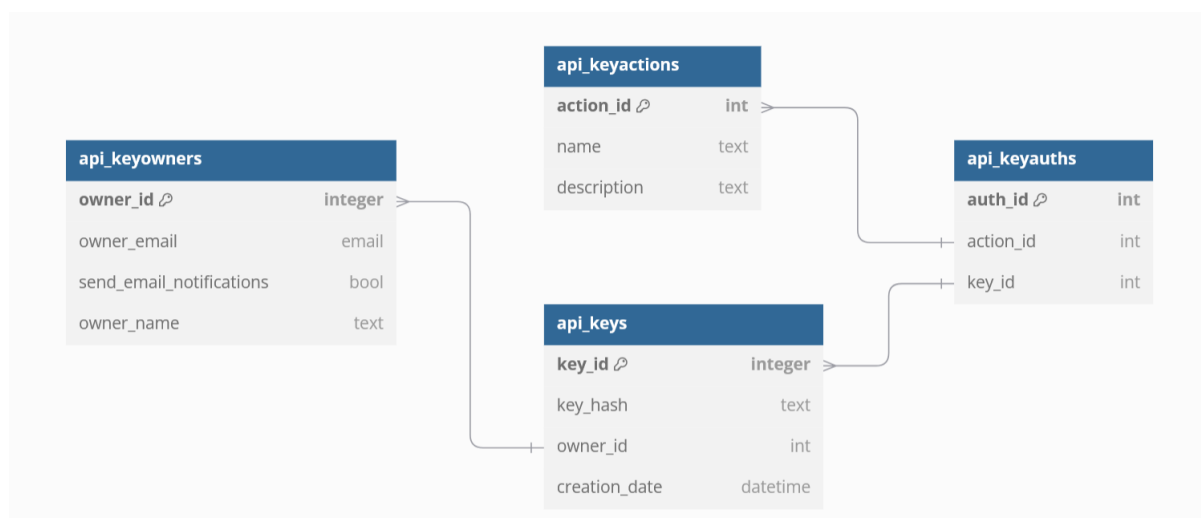
L'équipe recommande alors de ne pas changer ces codes.

api/models.py

Ce module contient un ensemble de classes décrivant l'architecture de la base de données de gestion de clés. Pour se faire, on utilise le système de “Models” de Django synchronisé à la base de données SQLite.

Les modèles créés par l'équipe incluent :

- Une table pour le stockage des informations sur les utilisateurs.
- Une table pour le stockage des clés d'API (stockées sous la forme de hashes).
- Une table pour le stockage des actions que peuvent réaliser les clés.
- Une table pour enregistrer les autorisations associées à chaque clé (lien entre une clé et une action).



Cette architecture peut être améliorée à l'aide du module `api/models.py` en modifiant le contenu des modèles associés à chacune des tables.

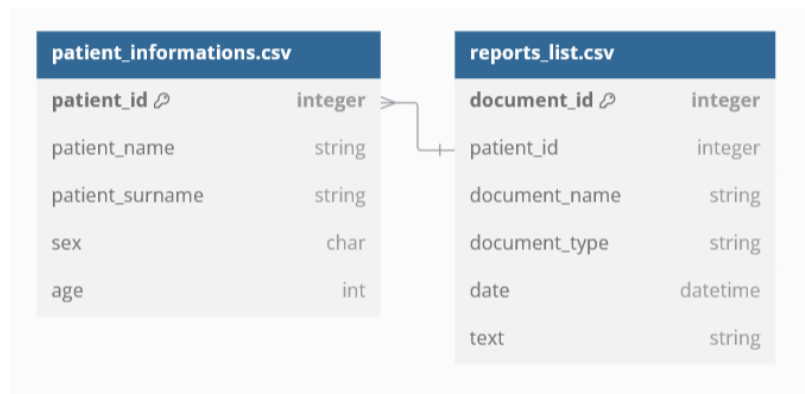
Les informations présentes dans les modèles peuvent être appliquées à l'aide de la procédure suivante :

- (Activer l'environnement virtuel contenant les librairies nécessaire au fonctionnement de l'API.)
- Se rendre à l'aide d'un terminal dans le dossier **api_interface** contenant le code de l'interface et de l'API (dont en particulier le code **manage.py**) à l'aide de la commande **cd**.
- Réaliser la commande **“python -m manage makemigrations”** pour préparer l'application des modifications.

- Réaliser la commande “**python -m manage migrate**” pour appliquer les modifications.

medical_database_example/interact_with_database.py

Ce module permet d’offrir une interface de communication avec la base de données médicale de test. Cette base de données n’est utilisée qu’avec l’IHM et a pour objectif de faciliter l’import de documents pour utiliser le module.



Architecture de la base de données utilisée pour les tests

get_patients_info(patient_id) -> (dict, bool)

Renvoie les informations de la base de données liées à un patient spécifique ou à l’ensemble des patients.

Arguments :

- patient_id : un entier optionnel permettant de récupérer les données d’un patient spécifique à partir de son identifiant dans la BD. Par défaut sa valeur est -1.

Return :

- Renvoie une liste avec les informations des patients ou du patient spécifique.

Example :

```
[
    {'name': 'Doe', 'surname': 'John', 'id': 1, 'sex': 'M', 'age': 24},
    ...
]
```

- Un booléen indiquant si on a trouvé les données du patient dont on a donné l'id (dans le cas où l'id passé en paramètre ne vaut pas -1).

`is_valid_patient_id(id) -> bool`

Permet de contrôler si un identifiant correspond à l'id d'un patient.

Arguments :

- `id (int)` : un identifiant à contrôler.

Return :

- `True` si l'identifiant est un identifiant patient, `False` sinon.

`get_documents(patient_id, document_id, id_exists) -> list[dict] | (list[dict], list[int])`

Permet de récupérer les données de la base de données de documents. Il est possible de récupérer tous les documents, les documents liés à un seul patient ou un seul document. Il est aussi possible de vérifier si des identifiants de documents existent dans la bd.

Arguments :

- `patient_id (int)` : identifiant de patient optionnel, dans le cas où l'on ne veut récupérer que les documents du patient d'identifiant "`patient_id`". (vaut -1 par défaut pour récupérer tous les documents).
- `document_id (int)` : identifiant de document optionnel, dans le cas où l'on ne veut récupérer qu'un seul document d'identifiant "`document_id`". (vaut -1 par défaut pour récupérer tous les documents).
- `id_exists (list[int] | None)` : liste d'entier optionnelle contenant des identifiants de

documents dont on veut récupérer l'index dans la liste finale.

Return :

- Une liste de dictionnaires contenant les informations des documents de la bd.

```
Output shape :  
[  
  {  
    'id': <document id : int>,  
    'patient_id': <patient id : int>,  
    'type': <document label : str>,  
    'text': <content of the document : str>,  
    'name': <document name : str>,  
    'date': <registering date : datetime.datetime>,  
    'patient_name': <name of the patient : str>,  
  },  
  ...  
]
```

- Dans le cas où `id_exists == None`, une liste indiquant l'index dans la liste des documents d'un document particulier avec son identifiant dans `id_exists` ou -1 si l'identifiant n'est pas valide.

document_name_used(doc_name) -> bool

Permet de vérifier si le nom d'un document est utilisé ou non.

Arguments :

- `doc_name (str)` : le nom d'un document.

Return :

- `True` si le nom est utilisé, `False` sinon.

documents_last_id() -> int

Renvoie le plus grand identifiant de document dans la base de données sous la forme d'un entier.

save_document(patient_id, document_name, document_type, document_content, imported_file) -> bool

Permet de sauvegarder un nouveau document dans la base de données.

Arguments :

- patient_id (int) : identifiant du patient rattaché au document.
- document_name (str) : nom du document.
- document_type (str) : label décrivant le type de document.
- document_content (str) : contenu du document.
- imported_file (file or None) : fichier de la base de donnée ouvert à l'aide de la commande "open" (pour éviter les doubles imports).

Return :

- Sauvegarde le nouveau fichier texte dans la base de données.
- Renvoie True si l'enregistrement est réussi.

delete_document(document_id) -> bool

Supprime un document de la base de donnée à partir de son identifiant.

Arguments :

- document_id (int) : l'identifiant du document à supprimer.

Return :

- 1 si le document a été supprimé, 0 sinon.

user interface/views.py

Ce module permet de gérer l'ensemble des requêtes en lien avec l'interface utilisateur. Ces fonctions se chargent principalement de récupérer les données nécessaires, d'utiliser les modules et de charger les pages de l'interfaces en y insérant les informations voulues par l'utilisateur.

api/views.py

Ce module permet de gérer l'ensemble des requêtes en lien avec l'API. Celui-ci se charge alors de contrôler les paramètres donnés par l'utilisateur, de contrôler les permissions, d'appeler les modules et de renvoyer les résultats/erreurs.

Les fonctionnalités permettant de traiter les requêtes liées à l'utilisation des modules étant déjà présentées dans la documentation utilisateur et ne se contentant que d'appeler les fonctions précédentes la liste de fonctions présentée ci-dessous ne détaille que le contenu des fonctions utilitaires pour le contrôle des données et des permissions.

process_params_form(request :WSGIRequest, inputs_names:list[str], value_constraints:list) -> list | dict:

Traite le paramètre envoyé dans une demande post-et renvoie une version traitée du dictionnaire des paramètres.

Renvoie une liste de clés invalides si les contraintes données dans le paramètre n'ont pas été respectées.

Arguments :

- request (WSGIRequest) : requête POST envoyée par l'utilisateur contenant les réponses de formulaire.
- inputs_names (list) : la liste des noms des champs à contrôler dans le formulaire.
- value_constraints : un descriptif de la contrainte que doit satisfaire chaque valeur dans les champs de inputs_names. Les valeurs de cette liste peuvent être :
 - "str not empty" : le paramètre doit être une chaîne de caractères non vide.
 - "positive int" : le paramètre doit être un entier positif.

- “positive float” : le paramètre doit être un flottant positif.
- “bool” : le paramètre doit être un booléen.
- “str list” : le paramètre doit être une liste de chaînes de caractères toutes non vides.
- “... optionnal” : le paramètre est optionnel.
- (valeur,1 valeur2,...) : le paramètre est obligatoire et doit nécessairement être une valeur du tuple.
- [valeur,1 valeur2,...] : le paramètre est facultatif et doit nécessairement être une valeur de la liste.

Return :

- si toutes les contraintes sont satisfaites la fonction renvoie un dictionnaire où
 - les clés sont les valeurs des inputs_names.
 - les valeurs sont les paramètres du formulaire qui ont été traités (str converti en int/float/bool)
- sinon la fonction renvoie une liste de valeurs d’inputs_names correspondant aux entrées du formulaire avec une valeur non valide.

request_inputs_error(processed_data:dict|list) -> JsonResponse | None :

Renvoie une erreur en cas de paramètres de formulaire non valides ou None si les paramètres sont valides.

Les entrées de cette fonction sont des sorties de la fonction process_params_form.

Arguments :

- processed_data : le résultat du traitement des données par la fonction process_params_form.

Return :

- une réponse JsonResponse avec une erreur "Invalid parameters" et la liste des champs invalides si les paramètres sont incorrects.

```
{
  "status" : "error",
  "type": "Invalid parameters.",
  "content" : {
    "invalid_fields" : list,
  }
}
```

- None si les paramètres sont valides.

`can_make_action(key_hash:str, action:str) -> bool`

Contrôle si une clé est utilisée comme autorisation pour effectuer une action donnée.

Arguments :

- `key_hash` : une version hachée de la clé.
- `action` : le nom de l'action à contrôler.

Return :

- True si l'utilisateur peut réaliser l'action, False sinon.

`can_make_action_with_key_str(key_str:str, action_str:str="Manage keys") -> bool`

Même fonction que la précédente mais prend en paramètre une clé non hashée.

`can_make_action_with_key_id(key_id:int, action:str) -> bool`

Même fonction que la précédente mais prend en paramètre l'id d'une clé.

generate_first_key.py

Ce module permet de générer une première clé d'API afin de pouvoir utiliser l'API et l'interface utilisateur. Il suffit alors de l'exécuter depuis un terminal python afin d'obtenir une clé.

Commentaires

Le rôle de l'interface et de l'API sont de jouer l'intermédiaire entre les modules et l'utilisateur de manière à rendre plus simple l'utilisation des modules. Les tests des deux fonctionnalités en temps réel montrent que ces solutions sont fonctionnelles mais font face aux mêmes difficultés que les modules qu'elles appellent.

Le système de contrôle des permissions n'a pas été équipé de système efficace pour empêcher le brute-force d'une clé d'API. Actuellement, on se contente d'attendre 3 secondes avant la vérification afin de limiter le nombre de tentatives réalisables dans un temps donné. Un contrôle plus poussé des requêtes pourrait prévenir le brute force (par exemple en autorisant à un utilisateur, identifié par les informations présentes dans le header de sa requête, qu'un certain nombre de requêtes).

L'interface a été testée à partir d'une petite base de données médicale basique n'ayant pour but que de démontrer ce qui est possible. Cependant, la modification du code du module `medical_database_example/interact_with_database.py` sans changer les signature des fonctions pourrait permettre de connecter facilement l'interface à un entrepôt de données médicales plus poussé.